

Basi di Dati

Nicolò Giuliani

Indice

1	Introduzione	5
1.1	Sistema informativo e sistema informatico	5
1.2	Informazione e dati	5
1.3	DBMS	5
1.4	Transazioni	6
1.5	Evoluzione storica	7
1.6	DBMS e file system	7
1.7	Modelli dei dati	7
1.8	Architettura di un DBMS	8
1.9	Linguaggi per basi di dati	9
1.10	Attori e utenti	9
1.11	Pregi e difetti dei DBMS	10
2	Il modello relazionale	10
2.1	Modelli logici	10
2.2	Il modello relazionale	10
2.3	Relazione logica	11
2.4	Struttura non posizionale	11
2.5	Modello value based	11
2.6	Schemi e istanze	12
2.7	Casi particolari	12
2.8	Informazione parziale	12
2.9	Vincoli di integrità	13
2.10	Vincoli di tupla	13
2.11	Chiavi	14
2.12	Chiavi e valori nulli	15
2.13	Integrità referenziale	15
2.14	Azioni compensative	15
3	Algebra relazionale e calcolo relazionale	16
3.1	Linguaggi di interrogazione	16
3.2	Algebra relazionale	16
3.3	Operatori insiemistici	17
3.4	Ridenominazione	17
3.5	Selezione	17
3.6	Proiezione	18
3.7	Selezione e proiezione	18
3.8	Join naturale	18
3.9	Cardinalità del join	19

3.10	Join esterno	20
3.11	Prodotto cartesiano e theta-join	20
3.12	Esempi di interrogazioni	21
3.13	Equivalenza di espressioni	21
3.14	Selezione con valori nulli	23
3.15	Viste	23
3.16	Notazione alternativa per i join	24
3.17	Calcolo relazionale	24
3.18	Calcolo su domini	25
3.19	Calcolo su tuple con dichiarazioni di range	27
3.20	Limiti di algebra e calcolo	28
3.21	Datalog	28
4	SQL di base	30
4.1	SQL	30
4.2	Storia	30
4.3	Definizione dei dati (DDL)	30
4.4	Tipi di dato	31
4.5	Vincoli intrarelazionali	31
4.6	Vincoli interrelazionali	32
4.7	Modifica degli schemi	32
4.8	Indici	33
4.8.1	Strutture fisiche di accesso	33
4.8.2	Classificazione degli indici	34
4.8.3	Alberi B e B+	34
4.8.4	Strutture hash	35
4.8.5	Costo delle operazioni	35
4.9	Interrogazioni: SELECT	35
4.10	Selezione e proiezione	36
4.11	SQL e algebra relazionale	37
4.12	Join	38
4.13	Join esterno	39
4.14	Ordinamento	39
4.15	Unione, intersezione, differenza	39
4.16	Interrogazioni annidate	40
4.17	Quantificatore esistenziale	42
4.18	Funzioni aggregate	42
4.19	Raggruppamento	43
4.20	Modifica dei dati	44
5	SQL avanzato	45
5.1	Vincoli CHECK	45
5.2	Asserzioni	46
5.3	Viste	46
5.4	Viste per query complesse	46
5.5	Interrogazioni ricorsive	47
5.6	Funzioni scalari	47
5.7	Sicurezza: privilegi	48
5.8	RBAC	49
5.9	Transazioni	50

5.10	Controllo di concorrenza: anomalie	50
5.11	Schedule e serializzabilità	51
5.12	Lock e locking a due fasi (2PL)	51
5.13	Livelli di isolamento in SQL	52
5.14	Affidabilità: log e ripresa dai guasti	52
5.15	ARIES	53
6	Modellazione concettuale dei dati	53
6.1	Perché i modelli concettuali	53
6.2	Progettazione di basi di dati	53
6.3	Fasi della progettazione	54
6.4	Il modello Entity-Relationship	55
6.5	Entità	55
6.6	Relazioni (relationship)	55
6.7	Attributi	56
6.8	Cardinalità delle relazioni	56
6.9	Cardinalità degli attributi	57
6.10	Identificatori	57
6.11	Generalizzazioni	58
6.12	Documentazione degli schemi concettuali	59
6.13	Modellazione dei dati in UML	60
7	Progettazione concettuale	61
7.1	Requisiti e progettazione concettuale	61
7.2	Requisiti	62
7.3	Esempi di requisiti	62
7.4	Dai requisiti ai costrutti ER	63
7.5	Pattern di progettazione	64
7.6	Strategie di progettazione	65
7.7	Metodologia	65
7.8	Esempio completo: società di formazione	66
8	Progettazione logica	66
8.1	Obiettivo	66
8.2	Ristrutturazione: prestazioni	67
8.3	Analisi delle ridondanze	68
8.4	Eliminazione delle generalizzazioni	69
8.5	Partizionamento e accorpamento	69
8.6	Scelta degli identificatori primari	70
8.7	Traduzione nel modello relazionale	70
8.8	Esempio completo	71
9	Normalizzazione	72
9.1	Forme normali	72
9.2	Una relazione con anomalie	72
9.3	Dipendenze funzionali	73
9.4	Forma normale di Boyce e Codd (BCNF)	73
9.5	Decomposizione senza perdita	74
9.6	Conservazione delle dipendenze	74
9.7	Terza forma normale (3NF)	75
9.8	Teoria delle dipendenze	76

9.9	Progettazione e normalizzazione	76
10	Basi di dati attive	77
10.1	Da passive ad attive	77
10.2	Evoluzione	77
10.3	Trigger	78
10.4	Modello computazionale	78
10.5	Trigger in Oracle	78
10.6	Trigger in DB2	80
10.7	Estensioni	81
10.8	Proprietà delle regole	81
10.9	Applicazioni	81

1 Introduzione

1.1 Sistema informativo e sistema informatico

Base di dati (significato generico): insieme organizzato di dati che supporta lo svolgimento delle attività specifiche di un ente (istituzione, azienda, ufficio, persona).

Base di dati (significato specifico): insieme di dati gestito da un DBMS.

Sistema informativo

componente di un ente che gestisce le informazioni di interesse. Ogni ente ha il proprio sistema informativo, anche se non esplicitamente identificabile nella sua organizzazione; va studiato nel suo contesto operativo. È un concetto *indipendente* dall'automazione: esistono da secoli organizzazioni la cui unica missione è gestire dati (anagrafi, banche).

Sistema informatico

la porzione automatizzata del sistema informativo, cioè la componente che gestisce le informazioni con tecnologie informatiche.

Gerarchia di inclusione: Azienda $\supset$ Organizzazione $\supset$ Sistema informativo $\supset$ Sistema informatico.

Gestione dell'informazione: raccolta/acquisizione, archiviazione/conservazione, elaborazione/trasformazione/produzione, distribuzione/comunicazione/scambio.

1.2 Informazione e dati

Le attività umane rappresentano l'informazione in molti modi (idee informali, linguaggio naturale, disegni e diagrammi, numeri e codici) su supporti diversi (mente umana, carta, dispositivi elettronici). Nei sistemi informatici invece l'informazione è rappresentata in modo povero ma essenziale: come **dati**.

- *informazione*: fatti forniti o appresi su qualcosa o qualcuno.
- *dato*: elemento di informazione, punto di partenza fissato di un procedimento.

Il dato è il risultato del processo di organizzazione, codifica e memorizzazione dell'informazione. **Il dato da solo è inutile senza la sua interpretazione** (es. i numeri su un cartello stradale non dicono nulla se non si sa che sono orari, e quale orario vale in quale giorno).

Perché i dati? È difficile rappresentare con precisione informazione e conoscenza ricche. I dati sono una *risorsa strategica* perché sono più stabili delle altre componenti (processi aziendali, tecnologie, ruoli): es. i dati di banche e anagrafi.

1.3 DBMS

DBMS (DataBase Management System): sistema che gestisce collezioni di dati

- **grandi**
- **persistenti**
- **condivise**

garantendo

- **privatezza**
- **affidabilità**
- **efficienza**
- **efficacia**

Esempi: DB2, Oracle, SQLServer, MySQL, PostgreSQL, Access, BigQuery, SQLite.

Grandi dimensioni molto maggiori della memoria centrale; il solo limite considerato è quello fisico dei dispositivi. Ordini di grandezza: 1–5 TB (dati transazionali), 30–50 TB (dati decisionali), 500–800 TB (dati scientifici), 10^{11} record.

Persistenti il loro tempo di vita è indipendente da quello dei processi che li usano.

Condivise ogni (grande) azienda è divisa in settori, ognuno con un proprio sotto-sistema informativo. Senza condivisione nascono **ridondanza** (stessi dati ripetuti) e quindi possibile **incoerenza** (descrizioni diverse non coincidono). La base di dati è una risorsa *integrata e condivisa*, da cui la necessità di **meccanismi di autorizzazione** (attività diverse sugli stessi dati) e **controllo della concorrenza** (più utenti sugli stessi dati).

Privatezza si definiscono autorizzazioni del tipo “l’utente A può leggere tutti i dati e modificare il dato X”.

Affidabilità resistenza a malfunzionamenti hardware e software. La base di dati è una risorsa preziosa e va conservata a lungo termine. Tecnica fondamentale: la gestione delle **transazioni**.

Efficienza uso proficuo delle risorse di memoria (primaria e secondaria) e di tempo (esecuzione e risposta). Poiché i DBMS offrono moltissime funzioni, potrebbero risultare inefficienti: da qui grandi investimenti e concorrenza fra prodotti. L’efficienza è anche frutto della qualità del software.

Efficacia rendere produttive le attività degli utenti, offrendo funzionalità potenti e flessibili.

1.4 Transazioni

Una **transazione** è un insieme di operazioni non separabili (*atomiche*), corrette anche in presenza di concorrenza, con effetti definitivi.

Atomicità

o si esegue tutta la sequenza o non se ne esegue nulla. Es.: “sposta denaro dal conto A al conto B: o si preleva da A e si versa su B, o non si fa nulla”.

Concorrenza

transazioni concorrenti devono restare coerenti: due assegni sullo stesso conto incassati contemporaneamente non devono perdersi; due agenzie non devono prenotare due volte lo stesso posto sul treno.

Durabilità

il **commit** garantisce che il risultato finale sia registrato e conservato, anche in caso di concorrenza o guasti.

Due punti di vista sulle transazioni:

- *utente*: un programma che realizza una funzione utile, frequente e predefinita, con poche eccezioni previste (versamento in conto, emissione di un certificato di nascita, iscrizione all'anagrafe, prenotazione di un volo). Sono scritte in un linguaggio ospite, noto o *ad hoc*.
- *sistema*: sequenza indivisibile di operazioni (cfr. affidabilità).

1.5 Evoluzione storica

Anni '70 — nessun DBMS

le applicazioni accedono direttamente ai file del sistema operativo; i dati comuni vengono duplicati nelle applicazioni.

Anni '80 — primi DBMS

fra le applicazioni (Pascal, C) e i file si interpone il DBMS, che offre tabelle.

Anni '90 — DBMS attivi

il comportamento procedurale comune viene condiviso fra le applicazioni. Nascono le **stored procedures**, che però non sono standardizzate e soffrono di *impedance mismatch* con il linguaggio usato per esprimerle; si introducono quindi i **trigger**, regole che modellano il comportamento procedurale condiviso e sono gestite dal DBMS stesso.

Anni 2000

architettura client/server su Internet: front-end (Javascript) sul client, Web Application Server e back-end (Java), database attivo con tabelle e trigger sul server.

Anni 2010

app mobili: front-end in Java, Objective-C, HTML5; il resto dell'architettura resta invariato.

1.6 DBMS e file system

Anche un file system può gestire grandi collezioni di dati, ma offre un accesso a grana grossa: “tutto o niente”. I DBMS estendono le funzionalità del file system, offrendo più servizi in modo integrato.

Descrizione dei dati ogni programma che non usa un DBMS contiene al proprio interno la descrizione della struttura dei file che tratta: programmi diversi possono avere descrizioni diverse, generando incoerenza fra le rappresentazioni. Nei DBMS una porzione della base di dati (il **catalogo** o *dizionario*) contiene una descrizione centralizzata dei dati, condivisa da tutte le applicazioni.

1.7 Modelli dei dati

Nei DBMS i dati sono descritti a diversi **livelli di astrazione**: i programmi usano una rappresentazione di alto livello indipendente da quella di livello più basso, così una modifica al livello basso non richiede di modificare i programmi. Questo è ottenuto attraverso il concetto di **modello dei dati**.

Modello dei dati: insieme di costrutti utilizzati per organizzare i dati e descriverne il comportamento. Componente fondamentale: dare struttura ai dati tramite **costruttori di tipo**. Es.: il **modello relazionale** fornisce il costruttore *relazione*, che permette di definire insiemi di record omogenei.

Schema e istanza in ogni base di dati esistono

- lo **schema**: invariante nel tempo, descrive la struttura dei dati (aspetto *intensionale*) — es. l'intestazione delle tabelle;
- l'**istanza**: i valori effettivi, che possono cambiare rapidamente (aspetto *estensionale*) — es. il corpo delle tabelle.

Due tipologie di modelli

Modelli logici

usati dai DBMS per organizzare e memorizzare i dati; usati dai programmi, indipendenti dalla rappresentazione fisica. Es.: relazionale, reticolare, gerarchico, a oggetti, XML.

Modelli concettuali

rappresentano i dati indipendentemente da ogni sistema specifico; descrivono concetti del mondo reale e si usano nella fase preliminare di progettazione. Il più diffuso è **Entity-Relationship**.

1.8 Architettura di un DBMS

Architettura semplificata (a due livelli): utente $\rightarrow$ *schema logico* $\rightarrow$ *schema interno* $\rightarrow$ DB.

Schema logico

descrive la struttura della base di dati tramite il modello logico dei dati (es. la struttura delle tabelle).

Schema interno (o fisico)

rappresenta lo schema logico a livello di memorizzazione, con strutture dati *raw* (file, record, puntatori).

Poiché la rappresentazione logica è indipendente da quella fisica — una tabella si usa “così com'è”, indipendentemente dalla rappresentazione sottostante, che può anche variare nel tempo — nel corso si considera solo il livello logico.

Architettura standard ANSI/SPARC a tre livelli aggiunge sopra lo schema logico uno o più **schemi esterni**, ciascuno usato da un gruppo di utenti:

- **schema logico**: struttura della base di dati secondo il modello logico “principale”;
- **schema interno**: rappresentazione a livello di memorizzazione;
- **schema esterno**: descrive una parte della base di dati in un modello logico (*viste* parziali, tabelle derivate, anche da modelli diversi).

Indipendenza dei dati è conseguenza della stratificazione; l'accesso ai dati avviene sempre a livello esterno (che può coincidere con quello logico).

Indipendenza fisica

livello logico ed esterno sono indipendenti dal livello fisico: una relazione è usata allo stesso modo qualunque sia la sua realizzazione fisica, che può cambiare senza modificare i programmi.

Indipendenza logica

il livello esterno è indipendente da quello logico: si può operare su una vista senza modificare il livello logico, e modifiche “trasparenti” allo schema logico lasciano inalterato lo schema esterno.

1.9 Linguaggi per basi di dati

Un altro aspetto dell'efficacia è la disponibilità di linguaggi e interfacce diverse:

- linguaggi testuali *interattivi* (**SQL**);
- comandi SQL immersi in un linguaggio **ospite** (Pascal, Java, C);
- comandi SQL immersi in un linguaggio *ad hoc* con altre funzionalità (grafici, stampa di tabelle) — es. Oracle PL/SQL;
- interfacce amichevoli, senza linguaggio testuale.

Esempio SQL interattivo date le tabelle Corsi(Corso, Docente, Aula) e Aule(Nome, Edificio, Piano), per ottenere i corsi che si tengono al piano terra:

```
SELECT Corso, Aula, Piano
FROM Aule, Corsi
WHERE Nome = Aula
      AND Piano = 'Terra'
```

Separazione fra dati e programmi

DML

data manipulation language: interrogazione e aggiornamento delle *istanze* della base di dati.

DDL

data definition language: definizione degli *schemi* (logico, esterno, fisico) e altre operazioni. Es.:

```
CREATE TABLE orario (
  corso      CHAR(20),
  docente    CHAR(20),
  aula       CHAR(4),
  ora        CHAR(5) )
```

1.10 Attori e utenti

- progettisti e realizzatori di DBMS;
- progettisti della base di dati e amministratori (**DBA**);
- progettisti e programmatori di applicazioni;
- utenti:
 - *utenti finali* (terminalisti): eseguono operazioni predefinite (transazioni);
 - *utenti casuali*: eseguono operazioni non previste a priori, usando linguaggi interattivi.

DBA persona o gruppo responsabile del controllo e della gestione centralizzata della base di dati (efficienza, affidabilità, autorizzazioni). Spesso il DBA si occupa anche della progettazione, tranne che nei progetti complessi.

1.11 Pregi e difetti dei DBMS

Pregi

- dati come risorsa comune, base di dati come modello della realtà;
- gestione dei dati centralizzata, standardizzabile ed economia di scala;
- disponibilità di servizi integrati;
- riduzione di ridondanze e incoerenze;
- indipendenza dei dati (favorisce sviluppo e manutenzione delle applicazioni).

Difetti

- costo dei prodotti e della loro adozione;
- funzionalità non separabili (con conseguente riduzione di efficienza).

2 Il modello relazionale

2.1 Modelli logici

Esistono tre modelli logici *tradizionali*:

- **gerarchico**;
- **reticolare** (network);
- **relazionale**.

Più recenti sono il modello a *oggetti* (poco diffuso) e i modelli su *XML* (considerati complementari al relazionale).

Caratteristica distintiva

Gerarchico e reticolare

usano **riferimenti espliciti** (puntatori) fra i record.

Relazionale

è **value based**: i riferimenti fra dati memorizzati in relazioni diverse sono rappresentati tramite *valori* (non puntatori).

2.2 Il modello relazionale

Definito da **E. F. Codd** nel 1970 per garantire l'indipendenza dei dati. Implementato in DBMS reali solo a partire dal 1981: realizzare indipendenza dei dati in modo efficiente e affidabile non è banale. Il modello si basa sulla definizione logica di *relazione* (con alcune differenze) e le relazioni sono rappresentate tramite **tabelle**.

Relazione, relation, relationship tre concetti da tenere distinti:

- *relazione logica*: nel senso della teoria degli insiemi;
- *relazione* nel modello relazionale;
- *relationship*: esprime una specifica classe di fatti nel modello Entity-Relationship.

2.3 Relazione logica

Dati n insiemi $D_1, \dots, D_n$ (non necessariamente distinti):

- il **prodotto cartesiano** $D_1 \times \dots \times D_n$ è l'insieme di tutte le n -uple $(d_1, \dots, d_n)$ tali che $d_1 \in D_1, \dots, d_n \in D_n$;
- una **relazione logica** su $D_1, \dots, D_n$ è un *sottoinsieme* $r \subseteq D_1 \times \dots \times D_n$;
- i D_i sono i **domini** della relazione.

Proprietà

- è un *insieme* di n -uple: non c'è ordine fra le tuple;
- le tuple sono tutte *distinte*;
- ogni n -upla è *ordinata*: l' i -esimo valore proviene dall' i -esimo dominio.

Struttura posizionale in `Matches` $\subseteq \text{string} \times \text{string} \times \text{int} \times \text{int}$ lo stesso dominio compare con **ruoli** diversi, distinti solo dalla *posizione* (1^a colonna = squadra di casa, 2^a = ospite, ecc.). Questa è una struttura **posizionale**.

2.4 Struttura non posizionale

Nel modello relazionale a ciascun dominio si associa un **attributo**, un nome unico che ne esprime il “ruolo”. La posizione delle colonne diventa irrilevante: la struttura è **non posizionale**. Es. (`Home`, `Away`, `GoalsH`, `GoalsA`): scambiando l'ordine delle colonne (con le relative intestazioni) l'informazione non cambia; scambiare *solo* le intestazioni cambia invece il significato.

Tabelle e relazioni una tabella rappresenta una relazione se

- tutte le righe sono *diverse*;
- tutte le intestazioni di colonna sono *diverse*;
- i valori all'interno di ogni colonna sono *omogenei* (stesso dominio).

Righe e colonne possono assumere qualunque posizione.

2.5 Modello value based

I riferimenti fra dati in relazioni diverse sono espressi tramite **valori** nelle tuple (es. `Exams.Student = Students.Number`), non tramite puntatori. Altri modelli (reticolare, gerarchico, a oggetti) usano invece riferimenti espliciti gestiti dal sistema.

Pregi del modello value based

- indipendenza dalla struttura fisica, che può cambiare dinamicamente (una cosa analoga si può ottenere con puntatori “di alto livello”);
- si memorizza solo ciò che è rilevante dal punto di vista applicativo;
- utente e programmatore vedono gli stessi dati;
- i dati possono essere facilmente condivisi fra ambienti diversi;
- i puntatori sono direzionali, i valori no.

2.6 Schemi e istanze

Schema di una relazione un nome di relazione R con un insieme di attributi $A_1, \dots, A_n$:

$$R(A_1, \dots, A_n)$$

Es. STUDENTS(Number, Surname, Name, Year of Birth).

Schema di database relazionale insieme di schemi di relazione:

$$\mathcal{R} = \{R_1(X_1), \dots, R_k(X_k)\}$$

Tupla una tupla su un insieme di attributi X è un **mapping** che associa a ogni attributo $A \in X$ un valore nel dominio di A . Con $t[A]$ si indica il valore della tupla t sull'attributo A (es. $t[\text{Name}] = \text{Mario}$).

Istanze

- una **istanza di relazione** r è un insieme finito di tuple con schema $R(X)$;
- una **istanza di database relazionale** su $\mathcal{R} = \{R_1(X_1), \dots, R_n(X_n)\}$ è un insieme $r = \{r_1, \dots, r_n\}$ di istanze di relazione (r_i istanza di R_i).

2.7 Casi particolari

Relazioni con un solo attributo sono ammesse: es. WorkerStudents(Number), che elenca soltanto le matricole degli studenti lavoratori.

Strutture annidate molti dati reali (es. scontrini, fatture) sono naturalmente *annidati*: un'intestazione con vari campi ripetuti (righe di prodotto). Il modello relazionale è *piatto*: le strutture annidate si rappresentano con più relazioni collegate da valori. Es. Receipts(Number, Date, Total) e Courses(Receipt, Row, No, Description, Price). La rappresentazione scelta dipende dagli obiettivi: righe ripetute ammesse? ordine delle righe significativo? esistono più rappresentazioni possibili.

2.8 Informazione parziale

Il modello relazionale ha una struttura *rigida*: l'informazione è espressa in tuple che devono rispettare uno schema. Ma i dati reali possono non combaciare con il formato atteso (es. un "secondo nome" non sempre esistente).

Perché non usare valori speciali scegliere un valore del dominio (0, stringa vuota, "99", ...) per marcare l'assenza è pericoloso:

- potrebbero non esserci valori "inutilizzati" disponibili;
- quei valori potrebbero in seguito diventare significativi;
- bisognerebbe comunque tracciare a livello di software questi valori "non rappresentabili".

Valore NULL tecnica grezza ma efficace: introdurre un valore speciale **NULL** che indica l'assenza di un valore atteso e non appartiene al dominio. Formalmente, per ogni attributo A , $t[A]$ è o un valore in $\text{dom}(A)$ oppure **NULL**. Servono vincoli specifici per stabilire quando **NULL** non è ammesso.

Tipi di NULL ci sono almeno tre casi diversi di valore mancante:

- valore *sconosciuto* (esiste ma non lo conosciamo);
- valore *inesistente* (non esiste per quella tupla);
- valore *non informativo* (non sappiamo se esiste).

I DBMS moderni non distinguono fra questi casi.

Troppi NULL una istanza con troppi NULL (specialmente nelle chiavi o nei riferimenti) perde valore informativo: l'uso di NULL va limitato.

2.9 Vincoli di integrità

Alcune istanze sintatticamente corrette possono non rappresentare informazione corretta per l'applicazione (es. un voto "32", una *laude* con voto < 30 , un riferimento a uno studente inesistente).

Vincolo di integrità proprietà che deve valere sulle istanze per rappresentare informazione corretta. Formalmente è una funzione booleana (**predicato**) che associa a ogni istanza *true* o *false*.

Perché

- descrizione più accurata del dominio reale;
- supporto alla *data quality*;
- utili in fase di progettazione;
- usati nella valutazione delle interrogazioni.

Osservazione i DBMS non supportano tutti i tipi di vincolo: si dichiarano quelli supportati e il DBMS ne impedisce la violazione; per gli altri il programmatore deve implementarli fuori dal DBMS.

Classificazione

Intra-relazionali

coinvolgono una sola relazione:

- su valori (*domain constraints*);
- su tuple.

Inter-relazionali

coinvolgono più relazioni.

2.10 Vincoli di tupla

Esprimono regole sui valori di ogni singola tupla, indipendentemente dalle altre. Caso specifico: i **domain constraints** coinvolgono un solo attributo.

Sintassi espressione booleana di atomi che confrontano:

- valori del dominio degli attributi, es. $(\text{Grade} \geq 18) \text{ AND } (\text{Grade} \leq 30)$;
- espressioni aritmetiche su tali valori, es. $\text{GrossPay} = (\text{Deductions} + \text{Net})$.

Una tupla (Bruni, 50.000, 11.000, 36.000) viola il vincolo perché $11.000 + 36.000 \neq 50.000$.

2.11 Chiavi

Nel mondo reale ogni entità è identificabile: nella tabella **Students** ogni tupla ha un **Number** unico e non esistono tuple con stessi **Surname**, **Name**, **Date of Birth**. Formalizziamo:

Superchiave

un insieme K di attributi di r tale che non esistono due tuple distinte $t_1, t_2 \in r$ con $t_1[K] = t_2[K]$.

Chiave

una superchiave *minimale*: nessun suo sottoinsieme proprio è già superchiave.

Esempi in **Students**(Number, Surname, Name, Curriculum, Birth):

- **Number** è chiave: è superchiave (identifica ciascuna tupla) e contiene un solo attributo, quindi è minimale;
- {Surname, Name, Birth} è un'altra chiave: è superchiave ed è minimale;
- {Surname, Curriculum} potrebbe essere superchiave in un'istanza particolare ma non nel caso generale.

Vincoli, schema e stati

- i vincoli corrispondono a proprietà del mondo reale, modellate nel DB;
- sono modellati sullo *schema* (si applicano a tutte le tuple);
- ogni schema può avere un insieme di vincoli: le tuple memorizzate sono **corrette** (valide) se soddisfano tutti i vincoli;
- un'istanza può soddisfare *per caso* altri vincoli non dichiarati.

Esistenza delle chiavi ogni relazione:

- non può contenere tuple con tutti i valori uguali (è un insieme);
- ha come superchiave l'insieme di tutti i suoi attributi;
- quindi ha almeno una chiave.

Importanza

- l'esistenza di almeno una chiave garantisce l'*accessibilità* di ogni tupla;
- le chiavi permettono di correlare tuple fra relazioni diverse: il modello relazionale è *value based*.

2.12 Chiavi e valori nulli

Se una tupla contiene NULL sugli attributi della chiave, i suoi valori non permettono di:

- identificare la tupla;
- creare facilmente riferimenti esterni da altre relazioni.

L'uso di NULL sulle chiavi va limitato.

Chiave primaria fra le chiavi di una relazione se ne sceglie una (**primary key**) che *non ammette valori NULL*. Convenzionalmente si indica con la sottolineatura degli attributi nello schema.

2.13 Integrità referenziale

Le informazioni fra relazioni diverse sono correlate tramite valori comuni, in particolare valori di chiavi (primarie). Tali correlazioni devono essere *coerenti*: es. il codice del vigile in `Infringement.Policeman` deve corrispondere a un ID esistente in `Policemen`.

Vincolo di integrità referenziale (*foreign key*) fra gli attributi X di una relazione R_1 e una relazione R_2 : i valori di X in R_1 devono comparire come valori della chiave primaria di R_2 .

Esempio in un DB con `Infringement(Code, Date, Policeman, Country, Plate)`, `Policemen(ID, Surname, Name)`, `Car(Country, Plate, Surname, Name)`:

- `Policeman → Policemen(ID)`;
- `(Country, Plate) in Infringement → Car(Country, Plate)`.

Una tupla `Infringement` con `(Italy, AJ046EZ)`, mentre in `Car` `AJ046EZ` appartiene alla `France`, viola il vincolo.

Osservazioni

- ruolo cruciale nel modello value based;
- i vincoli possono essere *rilassati* in presenza di NULL;
- le violazioni causate da operazioni possono essere gestite tramite *azioni compensative*;
- attenzione ai vincoli definiti su più attributi: l'*ordine* degli attributi è rilevante (in `Crashes(Code, CountryA, PlateA, CountryB, PlateB)` servono due vincoli distinti verso `Car(Country, Plate)`, uno per (A, A) e uno per (B, B)).

2.14 Azioni compensative

Quando una modifica (es. cancellazione di una tupla referenziata) violerebbe un vincolo referenziale, il DBMS può reagire in modi diversi.

Comportamento standard

Restrict

rifiuta la cancellazione.

Azioni compensative possibili

Cascade

rimuove “a catena” le tuple referenzianti (es. eliminando il progetto XYZ da `Projects` si eliminano anche i dipendenti in `Employees` con `Project = XYZ`).

Set NULL

pone NULL nell'attributo referenziante (`Employees.Project` diventa NULL per chi lavorava a XYZ).

Set default

introduce un valore di default.

3 Algebra relazionale e calcolo relazionale

3.1 Linguaggi di interrogazione

I linguaggi per basi di dati operano sullo schema (**DDL**) o sui dati (**DML**); il DML comprende istruzioni di *interrogazione* (query) e di *aggiornamento* (inserimento e modifica dei dati).

Dichiarativi

specificano *che cosa* si vuole ottenere, cioè le proprietà del risultato.

Procedurali

specificano *come* il risultato viene ottenuto.

- **Algebra relazionale**: procedurale;
- **Calcolo relazionale**: dichiarativo (teorico, non implementato);
- **SQL** (Structured Query Language): parzialmente dichiarativo (implementato);
- **QBE** (Query by Example): dichiarativo (implementato).

3.2 Algebra relazionale

È definita da un insieme di **operatori** che agiscono su relazioni, producono relazioni come risultato e sono quindi *componibili* fra loro.

Operatori:

- unione, intersezione, differenza;
- ridenominazione (ρ);
- selezione (σ);
- proiezione (π);
- join (join naturale, prodotto cartesiano, theta-join).

3.3 Operatori insiemistici

Le relazioni sono insiemi e anche i risultati devono esserlo: si possono usare **unione** ($\cup$), **intersezione** ($\cap$) e **differenza** ($-$), ma solo fra relazioni con lo *stesso schema*.

Graduated			Technicians		
Number	Name	Age	Number	Name	Age
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

- Graduated $\cup$ Technicians: 4 tuple (7274, 7432, 9824, 9297) — le tuple comuni compaiono una volta sola;
- Graduated $\cap$ Technicians: le tuple 7432 e 9824;
- Graduated $-$ Technicians: la sola tupla 7274.

Un'unione sensata ma impossibile l'unione di Fatherhood (attributi Father, Child) e Motherhood (attributi Mother, Child) avrebbe senso (l'insieme dei genitori) ma gli schemi sono diversi: serve prima una ridenominazione.

3.4 Ridenominazione

Operatore *unario*: “cambia lo schema” lasciando inalterati i dati.

$$\rho_{\text{NewName} \leftarrow \text{OldName}}(\text{Relation})$$

rinomina l'attributo OldName in NewName. Si possono rinominare più attributi insieme.

Esempi

$$\rho_{\text{Parent} \leftarrow \text{Father}}(\text{Fatherhood}) \cup \rho_{\text{Parent} \leftarrow \text{Mother}}(\text{Motherhood})$$

produce una relazione (Parent, Child) con tutte le coppie genitore-figlio. Analogamente, con Employee(Surname, Office, Salary) e Worker(Surname, Factory, Wage):

$$\rho_{\text{Office, Pay} \leftarrow \text{Office, Salary}}(\text{Employee}) \cup \rho_{\text{Office, Pay} \leftarrow \text{Factory, Wage}}(\text{Worker})$$

ha schema (Surname, Office, Pay).

3.5 Selezione

Operatore *unario*. Il risultato:

- ha lo *stesso schema* dell'operando;
- contiene un *sottoinsieme* delle tuple dell'operando;
- contiene le sole tuple che soddisfano un **predicato**.

$$\sigma_{\text{Predicate}}(\text{Relation})$$

dove *Predicate* è un'espressione booleana valutata su ogni tupla della relazione.

Esempi su $\text{Employee}(\text{Number}, \text{Surname}, \text{Office}, \text{Salary})$:

- chi guadagna più di 50: $\sigma_{\text{Salary}>50}(\text{Employee})$;
- chi guadagna più di 50 e lavora a Milano: $\sigma_{\text{Salary}>50 \wedge \text{Office}='Milan'}(\text{Employee})$;
- chi ha il cognome uguale al nome della sede (confronto fra due attributi della stessa tupla): $\sigma_{\text{Surname}=\text{Office}}(\text{Employee})$.

3.6 Proiezione

Operatore *unario*. Il risultato:

- ha come schema un *sottoinsieme* degli attributi dell'operando;
- è costruito a partire da *tutte* le tuple dell'operando.

$$\pi_{\text{AttributeList}}(\text{Relation})$$

contiene tutte le tuple della relazione, ristrette agli attributi in *AttributeList*.

Cardinalità il risultato di una proiezione

- contiene *al più* tante tuple quante l'operando;
- può contenerne meno: restringendo gli attributi alcune tuple possono diventare uguali, e i duplicati vengono eliminati (una relazione è un insieme);
- se X è una **superchiave** di R , allora $\pi_X(R)$ contiene esattamente tante tuple quante R .

Es.: se **Employee** contiene due **Rossi** entrambi a **Rome**, $\pi_{\text{Surname}, \text{Office}}(\text{Employee})$ ha una tupla in meno; $\pi_{\text{Number}, \text{Surname}}(\text{Employee})$ invece le conserva tutte perché **Number** è chiave.

3.7 Selezione e proiezione

Sono operatori “ortogonali”:

- **selezione**: decomposizione *orizzontale* (sceglie righe);
- **proiezione**: decomposizione *verticale* (sceglie colonne).

Combinandole si estrae da una relazione solo l'informazione di interesse:

$$\pi_{\text{Number}, \text{Surname}}(\sigma_{\text{Salary}>50}(\text{Employee}))$$

Limite con questi soli operatori si estrae informazione da *una sola* relazione: non si possono correlare tuple di relazioni diverse, né tuple diverse della stessa relazione.

3.8 Join naturale

Il **join** è l'operatore più interessante dell'algebra relazionale: permette di correlare tuple di relazioni diverse. Es. in un concorso i compiti sono anonimi e associati a una busta chiusa col nome del candidato; compito e busta hanno lo stesso ID: da $(\text{ID}, \text{Grade})$ e $(\text{ID}, \text{Candidate})$ si ottiene $(\text{ID}, \text{Candidate}, \text{Grade})$.

Il join naturale è un operatore *binario* (generalizzabile a n operandi per associatività). Il risultato:

- ha come schema l'*unione* degli attributi dei due operandi;
- ogni sua tupla è ottenuta combinando due tuple, una per operando, che coincidono sugli attributi comuni.

Definizione dati $R_1(X_1)$ e $R_2(X_2)$, $R_1 \bowtie R_2$ è una relazione su X_1X_2 :

$$R_1 \bowtie R_2 = \{ t \text{ su } X_1X_2 \mid \exists t_1 \in R_1, t_2 \in R_2 \text{ con } t[X_1] = t_1 \text{ e } t[X_2] = t_2 \}$$

Casi

Employee	Dept		Dept	Chief		Employee	Dept	Chief
Rossi	A	$\bowtie$	A	Mori	$=$	Rossi	A	Mori
Neri	B		B	Bruni		Neri	B	Bruni
Bianchi	B					Bianchi	B	Bruni

Join completo

ogni tupla di entrambi gli operandi contribuisce al risultato (esempio sopra).

Join non completo

alcune tuple non trovano corrispondenza e restano escluse (tuple *dangling*): con **Dept** $\in \{B, C\}$ nel secondo operando, **Rossi/A** e il dipartimento **C** spariscono.

Join vuoto

nessuna tupla trova corrispondenza (dipartimenti **D**, **C** a destra): il risultato ha lo schema ma nessuna tupla.

Join con $n \times m$ tuple

se tutte le tuple hanno lo stesso valore sull'attributo comune, ogni tupla si combina con tutte quelle dell'altro operando.

3.9 Cardinalità del join

- In generale:

$$0 \leq |R_1 \bowtie R_2| \leq |R_1| \times |R_2|$$

- se il join coinvolge una **chiave** di R_2 :

$$0 \leq |R_1 \bowtie R_2| \leq |R_1|$$

perché i valori della chiave sono unici: ogni tupla di R_1 si combina con al più una tupla di R_2 (mentre una tupla di R_2 può combinarsi con più tuple di R_1);

- se il join coinvolge una chiave di R_2 e c'è un **vincolo di integrità referenziale** fra gli attributi di R_1 e la chiave di R_2 :

$$|R_1 \bowtie R_2| = |R_1|$$

perché ogni tupla di R_1 è associata a esattamente una tupla di R_2 .

Es.: $R_1(\text{Employee}, \text{Dept})$ e $R_2(\text{Room}, \text{Dept})$ con 3 tuple ciascuna, dove **Dept** non è chiave: il join può arrivare a $3 \times 3 = 9$ tuple (con due impiegati in B e due stanze in B se ne ottengono 4).

3.10 Join esterno

Nel join “normale” (*interno*) le tuple senza corrispondenza vengono scartate. Il **join esterno** (*outer join*) le conserva, completandole con valori NULL. Ne esistono tre varianti:

Sinistro ($\bowtie$)

mantiene tutte le tuple del *primo* operando, anche se non hanno corrispondenza nel secondo; gli attributi mancanti valgono NULL.

Destro ($\bowtie$)

mantiene tutte le tuple del *secondo* operando: $A \bowtie B$ equivale a $B \bowtie A$.

Completo ($\bowtie$)

mantiene tutte le tuple di entrambi: è l'unione del sinistro e del destro.

Employee		Department	
Employee	Dept	Dept	Chief
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Employee $\bowtie$ Department		
Employee	Dept	Chief
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

Il join sinistro contiene le prime tre righe, il destro le prime due e l'ultima.

3.11 Prodotto cartesiano e theta-join

Prodotto cartesiano è il join naturale fra due relazioni *senza attributi in comune*: ogni tupla del primo operando si combina con ogni tupla del secondo, quindi la cardinalità del risultato è il prodotto delle cardinalità. In pratica è utile solo se seguito da una selezione:

$$\sigma_{\text{condition}}(R_1 \bowtie R_2)$$

Theta-join questa combinazione ha un nome e una notazione propri:

$$R_1 \bowtie_{\text{condition}} R_2 = \sigma_{\text{condition}}(R_1 \bowtie R_2)$$

La condizione è di solito una congiunzione (AND) di atomi di confronto $A_1 \mathcal{R} A_2$, con $\mathcal{R}$ operatore di confronto ($=, >, <, \dots$).

Equi-join un theta-join in cui tutti gli atomi sono uguaglianze. È molto più usato del theta-join generale: permette di indicare su quali attributi fare il join senza ricorrere a ridenominazioni.

Es. con `Employee(Employee, Dept)` e `Department(Code, Chief)`:

$$\text{Employee} \bowtie_{\text{Dept}=\text{Code}} \text{Department}$$

ha schema `(Employee, Dept, Code, Chief)`. Il risultato è analogo a quello del join naturale con ridenominazione, che però ha un attributo in meno:

$$\text{Employee} \bowtie \rho_{\text{Dept} \leftarrow \text{Code}}(\text{Department}) \quad \text{schema}(\text{Employee}, \text{Dept}, \text{Chief})$$

3.12 Esempi di interrogazioni

Employee				Supervisor	
Number	Name	Age	Wage	Employee	Chief
7309	Rossi	34	45	7309	5698
5998	Bianchi	37	38	5998	5698
9553	Neri	42	35	9553	4076
5698	Bruni	43	42	5698	4076
4076	Mori	45	50	4076	8123
8123	Lupi	46	60		

1. Numero, nome, età e stipendio degli impiegati che guadagnano più di 40:

$$\sigma_{\text{Wage} > 40}(\text{Employee})$$

2. Numero, nome ed età degli impiegati che guadagnano più di 40:

$$\pi_{\text{Number, Name, Age}}(\sigma_{\text{Wage} > 40}(\text{Employee}))$$

3. Capi degli impiegati che guadagnano più di 40:

$$\pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} > 40}(\text{Employee}))$$

(risultato: 5698, 4076, 8123).

4. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\pi_{\text{Name, Wage}}(\text{Employee} \bowtie_{\text{Number=Chief}} \pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} > 40}(\text{Employee})))$$

5. Impiegati che guadagnano più del proprio capo, con numero, nome e stipendio di entrambi. **Employee** serve due volte (come impiegato e come capo), quindi una copia va ridenominata:

$$\pi_{\text{Number, Name, Wage, NoC, NameC, WageC}}(\sigma_{\text{Wage} > \text{WageC}}(\rho_{\text{NoC, NameC, WageC, AgeC} \leftarrow \text{Number, Name, Wage, Age}}(\text{Employee}) \bowtie_{\text{NoC=Chief}} (\text{Supervisor} \bowtie_{\text{Employee=Number}} \text{Employee})))$$

(risultato: Rossi, 45, con capo Bruni, 42).

6. Capi i cui impiegati guadagnano **tutti** più di 40. Il “per ogni” si esprime con la *differenza*: tutti i capi meno quelli che hanno almeno un impiegato che guadagna ≤ 40 :

$$\pi_{\text{Chief}}(\text{Supervisor}) - \pi_{\text{Chief}}(\text{Supervisor} \bowtie_{\text{Employee=Number}} \sigma_{\text{Wage} \leq 40}(\text{Employee}))$$

(risultato: 8123).

3.13 Equivalenza di espressioni

Due espressioni E_1 ed E_2 sono **equivalenti** se producono lo stesso risultato *qualunque sia* l'istanza della base di dati. Le regole di equivalenza sono importanti perché i DBMS riscrivono le interrogazioni in espressioni equivalenti ma più efficienti.

1. **Atomizzazione delle selezioni** (cascata di selezioni):

$$\sigma_{A>20}(\sigma_{B='Jones'}(R)) = \sigma_{A>20 \wedge B='Jones'}(R)$$

2. **Idempotenza delle proiezioni** (X, Y attributi di R):

$$\pi_X(R) = \pi_X(\pi_{XY}(R))$$

3. **Anticipazione della selezione** rispetto al join (*push selections*), con A attributo di R_2 :

$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie \sigma_{A=10}(R_2)$$

Riduce drasticamente la dimensione del risultato intermedio (e quindi il costo dell'operazione).

4. **Anticipazione della proiezione** rispetto al join (*push projections*), con $R_1(X_1)$, $R_2(X_2)$ e $Y_2 \subseteq X_2$:

$$\pi_{X_1 Y_2}(R_1 \bowtie R_2) = R_1 \bowtie \pi_{Y_2}(R_2)$$

Vale se $Y_2 \supseteq X_1 \cap X_2$, cioè se gli attributi eliminati ($X_2 - Y_2$) non sono coinvolti nel join.

5. **Definizione di theta-join**:

$$\sigma_{A=10}(R_1 \bowtie R_2) = R_1 \bowtie_{A=10} R_2$$

$$\sigma_{A=10 \wedge B='Jones'}(R_1 \bowtie R_2) = \sigma_{A=10}(R_1 \bowtie_{B='Jones'} R_2) = \sigma_{B='Jones'}(R_1 \bowtie_{A=10} R_2)$$

6–8. **Distributività**:

$$\sigma_{A=10}(R_1 \cup R_2) = \sigma_{A=10}(R_1) \cup \sigma_{A=10}(R_2)$$

$$\sigma_{A=10}(R_1 - R_2) = \sigma_{A=10}(R_1) - \sigma_{A=10}(R_2)$$

$$\pi_X(R_1 \cup R_2) = \pi_X(R_1) \cup \pi_X(R_2)$$

Attenzione: la proiezione *non* è distributiva rispetto alla differenza.

9–11. **Selezione e operatori insiemistici**:

$$\sigma_{A=10 \vee B='Jones'}(R) = \sigma_{A=10}(R) \cup \sigma_{B='Jones'}(R)$$

$$\sigma_{A=10 \wedge B='Jones'}(R) = \sigma_{A=10}(R) \cap \sigma_{B='Jones'}(R)$$

$$\sigma_{A=10 \wedge \neg(B='Jones')}(R) = \sigma_{A=10}(R) - \sigma_{B='Jones'}(R)$$

12. **Distributività del join rispetto all'unione**:

$$R \bowtie (R_1 \cup R_2) = (R \bowtie R_1) \cup (R \bowtie R_2)$$

Esempio di ottimizzazione numeri dei capi degli impiegati con meno di 30 anni (No = Number, Emp = Employee):

$$\pi_{\text{Chief}}(\sigma_{\text{No}=\text{Emp} \wedge \text{Age}<30}(\text{Employee} \bowtie \text{Supervisor}))$$

- regola 1, si spezza la selezione:

$$\pi_{\text{Chief}}(\sigma_{\text{No}=\text{Emp}}(\sigma_{\text{Age}<30}(\text{Employee} \bowtie \text{Supervisor})))$$

- regola 5 per No = Emp e regola 3 per Age < 30:

$$\pi_{\text{Chief}}(\sigma_{\text{Age}<30}(\text{Employee}) \bowtie_{\text{No}=\text{Emp}} \text{Supervisor})$$

- regola 4, si eliminano gli attributi inutili:

$$\pi_{\text{Chief}}(\pi_{\text{No}}(\sigma_{\text{Age}<30}(\text{Employee})) \bowtie_{\text{No}=\text{Emp}} \text{Supervisor})$$

Nota nel corso non è richiesto scrivere interrogazioni efficienti: conta che siano *corrette e chiare*, perché all'ottimizzazione pensa il DBMS.

3.14 Selezione con valori nulli

Una condizione atomica come $\text{Age} > 40$ è soddisfatta solo da valori *non nulli*: una tupla con $\text{Age} = \text{NULL}$ non compare nel risultato di $\sigma_{\text{Age} > 40}(\text{Employee})$.

Conseguenze indesiderate selezioni e condizioni atomiche sono valutate separatamente, quindi in presenza di NULL

$$\sigma_{\text{Age} > 30}(\text{People}) \cup \sigma_{\text{Age} \leq 30}(\text{People}) \neq \text{People} \quad \sigma_{\text{Age} > 30 \vee \text{Age} \leq 30}(\text{People}) \neq \text{People}$$

IS NULL / IS NOT NULL per riferirsi ai valori nulli si usano condizioni atomiche apposite (si potrebbe definire una logica a tre valori *true, false, unknown*, ma non è necessario):

$$\begin{aligned} & \sigma_{\text{Age} > 30}(\text{People}) \cup \sigma_{\text{Age} \leq 30}(\text{People}) \cup \sigma_{\text{Age IS NULL}}(\text{People}) \\ &= \sigma_{\text{Age} > 30 \vee \text{Age} \leq 30 \vee \text{Age IS NULL}}(\text{People}) = \text{People} \end{aligned}$$

Es.: la selezione

$$\sigma_{(\text{Age} > 40) \vee (\text{Age IS NULL})}(\text{Employee})$$

restituisce anche l'impiegato di cui non si conosce l'età.

3.15 Viste

Le viste sono rappresentazioni diverse degli stessi dati: corrispondono allo *schema esterno* dell'architettura ANSI/SPARC.

Relazioni di base

hanno contenuto autonomo (originale).

Relazioni derivate

il loro contenuto è definito tramite un'interrogazione; possono essere definite anche a partire da altre relazioni derivate (ma senza ricorsione).

Esempio con $\text{Concern}(\text{Employee}, \text{Dept})$ e $\text{Manage}(\text{Dept}, \text{Chief})$:

$$\text{Supervisor} = \pi_{\text{Employee}, \text{Chief}}(\text{Concern} \bowtie \text{Manage})$$

Due tipi di relazioni derivate

Viste materializzate

la relazione derivata è memorizzata fisicamente (in modo temporaneo o permanente).

- pro: i dati sono immediatamente disponibili per le interrogazioni;
- contro: ridondanza, aggiornamenti rallentati, raramente supportate dai DBMS.

Relazioni virtuali (o viste)

supportate da tutti i DBMS. Un'interrogazione su una vista viene trasformata in un'interrogazione sulla base di dati sottostante, sostituendo al nome della vista la sua definizione:

$$\sigma_{\text{Chief} = \text{'Leoni'}}(\text{Supervisor}) \rightsquigarrow \sigma_{\text{Chief} = \text{'Leoni'}}(\pi_{\text{Employee}, \text{Chief}}(\text{Concern} \bowtie \text{Manage}))$$

Perché le viste

- *schema esterno*: ogni utente vede ciò che gli interessa, nel modo in cui gli interessa, senza distrazioni, e solo ciò che è autorizzato a vedere;
- *strumento di programmazione*: semplificano la scrittura di interrogazioni complesse, soprattutto se ci sono sottoespressioni ripetute;
- permettono di usare programmi esistenti su schemi ristrutturati.

Le viste però *non* influiscono sull'efficienza delle interrogazioni.

Viste come strumento di programmazione impiegati che hanno lo stesso capo di Jones.

- senza vista:

$$\pi_{\text{Employee}}((\text{Concern} \bowtie \text{Manage}) \bowtie \rho_{\text{EmpR}, \text{DeptR} \leftarrow \text{Employee}, \text{Dept}}(\sigma_{\text{Employee}='Jones'}(\text{Concern} \bowtie \text{Manage})))$$

- con la vista **Supervisor**:

$$\pi_{\text{Employee}}(\text{Supervisor} \bowtie \rho_{\text{EmpR} \leftarrow \text{Employee}}(\sigma_{\text{Employee}='Jones'}(\text{Supervisor})))$$

Aggiornamento delle viste “aggiornare una vista” significa modificare le relazioni di base in modo che la vista, ricalcolata, rifletta l'aggiornamento. A ogni aggiornamento sulla vista deve corrispondere un aggiornamento sulle relazioni di base, ma questo può *non essere univoco*. Es.: inserire in **Supervisor** il fatto che Bruni è capo di Lupi è ambiguo se Bruni dirige due dipartimenti (in quale va Lupi?); inserire che Falchi è capo di Belli è impossibile senza inventare un dipartimento diretto da Falchi. Per questo sulle viste sono ammessi solo pochi tipi di aggiornamento.

3.16 Notazione alternativa per i join

È l'approccio adottato di solito in SQL:

- si ignora il join naturale (non si assumono implicitamente condizioni sugli attributi con lo stesso nome);
- con la sintassi **Relazione.Attributo** si distinguono gli attributi omonimi di relazioni diverse;
- le relazioni si “rinominano” definendo viste (assegnazione :=), e gli attributi si ridenominano solo quando serve per l'unione.

Es. (impiegati che guadagnano più del proprio capo), con $\text{Chief} := \text{Emp}$:

$$\pi_{\text{Emp.No}, \text{Emp.Name}, \text{Emp.Wage}, \text{Chief.No}, \text{Chief.Name}, \text{Chief.Wage}}(\sigma_{\text{Emp.Wage} > \text{Chief.Wage}}(\text{Chief} \bowtie_{\text{Chief.No} = \text{Sup.Chief}} (\text{Sup} \bowtie_{\text{Sup.Emp} = \text{Emp.No}} \text{Emp})))$$

3.17 Calcolo relazionale

Famiglia di linguaggi **dichiarativi** basati sulla logica del primo ordine. Due versioni:

- calcolo relazionale **su domini**;
- calcolo **su tuple con dichiarazioni di range**.

Negli esempi si usano le relazioni

Employee(Number, Name, Age, Salary) **Supervisor**(Chief, Employee)

3.18 Calcolo su domini

Un'espressione ha la forma

$$\{ A_1 : x_1, \dots, A_k : x_k \mid f \}$$

- f è una **formula** (con operatori booleani e quantificatori);
- $A_1 : x_1, \dots, A_k : x_k$ è la **target list**: $A_1, \dots, A_k$ sono attributi distinti (anche non presenti nella base di dati), $x_1, \dots, x_k$ variabili distinte.

Semantica: il risultato è una relazione su $A_1, \dots, A_k$ che contiene le tuple di valori per $x_1, \dots, x_k$ che rendono vera f .

Differenze rispetto al calcolo dei predicati

- i simboli di predicato corrispondono alle relazioni della base di dati, più i predicati “standard” predefiniti ($=, >, \dots$);
- non ci sono simboli di funzione;
- interessano soprattutto le formule *aperte* (con variabili libere);
- la notazione è *non posizionale*.

Esempi

0a. Numero, nome, età e stipendio di chi guadagna più di 40 ($\sigma_{\text{Salary} > 40}(\text{Employee})$):

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s \mid \\ \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

0b. Numero, nome ed età di tutti gli impiegati ($\pi_{\text{Number}, \text{Name}, \text{Age}}(\text{Employee})$):

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a \mid \\ \exists s (\text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s)) \}$$

Il quantificatore esistenziale sulle variabili che non compaiono nella target list si può sottintendere.

1. Numero, nome ed età di chi guadagna più di 40:

$$\{ \text{Number} : m, \text{Name} : n, \text{Age} : a \mid \\ \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

2. Capi degli impiegati che guadagnano più di 40 (il join si esprime usando la *stessa variabile* e nei due predicati):

$$\{ \text{Chief} : c \mid \text{Supervisor}(\text{Chief} : c, \text{Employee} : e) \\ \wedge \text{Employee}(\text{Number} : e, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \}$$

3. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\{ \text{NameC} : nc, \text{WageC} : sc \mid \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \wedge s > 40 \\ \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ \wedge \text{Employee}(\text{Number} : c, \text{Name} : nc, \text{Age} : ac, \text{Salary} : sc) \}$$

4. Impiegati che guadagnano più del proprio capo:

$$\begin{aligned} & \{ \text{No} : m, \text{Name} : n, \text{Wage} : s, \text{NoC} : c, \text{NameC} : nc, \text{WageC} : sc \mid \\ & \quad \text{Employee}(\text{Number} : m, \text{Name} : n, \text{Age} : a, \text{Salary} : s) \\ & \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ & \quad \wedge \text{Employee}(\text{Number} : c, \text{Name} : nc, \text{Age} : ac, \text{Salary} : sc) \\ & \quad \wedge s > sc \} \end{aligned}$$

5. Numero e nome dei capi i cui impiegati guadagnano *tutti* più di 40:

$$\begin{aligned} & \{ \text{Number} : c, \text{Name} : n \mid \text{Employee}(\text{Number} : c, \text{Name} : n, \text{Age} : e, \text{Salary} : s) \\ & \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m) \\ & \quad \wedge \neg \exists m' (\exists n' (\exists e' (\exists s' (\\ & \quad \quad \text{Employee}(\text{Number} : m', \text{Name} : n', \text{Age} : e', \text{Salary} : s') \\ & \quad \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m') \wedge s' \leq 40)))) \} \end{aligned}$$

Quantificatori e De Morgan esistenziale e universale sono intercambiabili:

$$\begin{aligned} \neg(A \wedge B) &= \neg A \vee \neg B & \neg(A \vee B) &= \neg A \wedge \neg B \\ \neg \exists x A &= \forall x \neg A & \neg \forall x A &= \exists x \neg A & \exists x A &= \neg \forall x \neg A \end{aligned}$$

Nell'esempio 5 la negazione dell'esistenziale si può quindi riscrivere come

$$\begin{aligned} & \forall m' (\forall n' (\forall e' (\forall s' (\\ & \quad \neg (\text{Employee}(\text{Number} : m', \text{Name} : n', \text{Age} : e', \text{Salary} : s') \\ & \quad \quad \wedge \text{Supervisor}(\text{Chief} : c, \text{Employee} : m')) \\ & \quad \vee s' > 40)))) \end{aligned}$$

Pregi e difetti

- pregio: è dichiarativo;
- difetto: è “verboso” (moltissime variabili);
- difetto: ammette espressioni senza senso, come

$$\{A : x \mid \neg R(A : x)\} \quad \{A : x, B : y \mid R(A : x)\} \quad \{A : x, B : y \mid R(A : x) \wedge y = y\}$$

il cui risultato dipende dal dominio delle variabili e non solo dall'istanza: sono espressioni **dipendenti dal dominio**, da evitare. In algebra relazionale non si possono scrivere, perché l'algebra è *indipendente dal dominio*.

Calcolo su domini e algebra sono “equivalenti”:

- per ogni espressione del calcolo indipendente dal dominio esiste un'espressione dell'algebra equivalente;
- per ogni espressione dell'algebra esiste un'espressione del calcolo equivalente e indipendente dal dominio.

3.19 Calcolo su tuple con dichiarazioni di range

Nasce per superare i limiti del calcolo su domini:

- ridurre il numero di variabili: le variabili denotano *tuple* (una variabile per ciascuna tupla coinvolta) anziché singoli valori;
- garantire che tutti i valori provengano dalla base di dati.

Sintassi

$$\{ TargetList \mid RangeList \mid Formula \}$$

- *RangeList* elenca le variabili libere della formula, indicando per ciascuna la relazione da cui proviene: $x(R)$;
- *TargetList* ha elementi della forma $Y : x.Z$ (oppure $x.Z$, o $x.*$ per tutti gli attributi);
- *Formula* è costruita con:
 - atomi di confronto $x.A \mathcal{R} c$ e $x.A \mathcal{R} y.B$;
 - connettivi logici;
 - quantificatori che associano un range alle variabili: $\exists x(R)(\dots)$, $\forall x(R)(\dots)$.

Esempi (le stesse interrogazioni del calcolo su domini)

0a. $\{ i.* \mid i(\text{Employee}) \mid i.\text{Salary} > 40 \}$

0b. $\{ i.(\text{Number}, \text{Name}, \text{Age}) \mid i(\text{Employee}) \mid \}$

1. $\{ i.(\text{Number}, \text{Name}, \text{Age}) \mid i(\text{Employee}) \mid i.\text{Salary} > 40 \}$
2. $\{ s.\text{Chief} \mid i(\text{Employee}), s(\text{Supervisor}) \mid i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > 40 \}$
3. Nome e stipendio dei capi degli impiegati che guadagnano più di 40:

$$\{ \text{NameC}, \text{WageC} : i'.(\text{Name}, \text{Salary}) \mid \\ i'(\text{Employee}), s(\text{Supervisor}), i(\text{Employee}) \mid \\ i'.\text{Number} = s.\text{Chief} \wedge i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > 40 \}$$

4. Impiegati che guadagnano più del proprio capo:

$$\{ i.(\text{Name}, \text{Number}, \text{Salary}), \text{NameC}, \text{NoC}, \text{WageC} : i'.(\text{Name}, \text{Number}, \text{Salary}) \mid \\ i'(\text{Employee}), s(\text{Supervisor}), i(\text{Employee}) \mid \\ i'.\text{Number} = s.\text{Chief} \wedge i.\text{Number} = s.\text{Employee} \wedge i.\text{Salary} > i'.\text{Salary} \}$$

5. Capi i cui impiegati guadagnano tutti più di 40:

$$\{ i.(\text{Number}, \text{Name}) \mid s(\text{Supervisor}), i(\text{Employee}) \mid i.\text{Number} = s.\text{Chief} \\ \wedge \neg(\exists i'(\text{Employee})(\exists s'(\text{Supervisor})(s.\text{Chief} = s'.\text{Chief} \\ \wedge s'.\text{Employee} = i'.\text{Number} \wedge i'.\text{Salary} \leq 40))) \}$$

Limite: l'unione questo calcolo non permette di esprimere alcune interrogazioni importanti, in particolare le **unioni** $R_1(AB) \cup R_2(AB)$: ogni variabile ha un solo range, mentre il risultato dovrebbe provenire da due relazioni diverse (né una né due variabili bastano). Intersezione e differenza invece si possono esprimere. Per questo SQL, che si basa su questo calcolo, ha un operatore esplicito di unione, mentre intersezione e differenza non sono presenti in tutte le implementazioni.

3.20 Limiti di algebra e calcolo

Algebra e calcolo sono sostanzialmente equivalenti ed esprimono un insieme significativo di operazioni, ma alcune interrogazioni utili restano fuori:

- si possono solo *estrarre* valori, non *calcolarne* di nuovi:
 - a livello di tupla (conversioni, somme, differenze, ...);
 - su insiemi di tuple (somme, medie, ...);

queste estensioni sono presenti in SQL;

- interrogazioni **ricorsive**, come la **chiusura transitiva**.

Chiusura transitiva data `Supervisor(Employee, Chief)`, trovare per ogni impiegato *tutti* i suoi superiori (il capo, il capo del capo, e così via).

Employee	Chief		Employee	Superior	
Rossi	Lupi		Rossi	Lupi	
Neri	Bruni		Neri	Bruni	
Lupi	Falchi	⇒	Lupi	Falchi	
Falchi	Leoni		Falchi	Leoni	
			Rossi	Falchi	(1 join)
			Lupi	Leoni	(1 join)
			Rossi	Leoni	(2 join)

Con un join della relazione con se stessa (più ridenominazione) si ottengono i superiori di secondo livello, con due join quelli di terzo, e così via: il numero di join necessari dipende dall'*istanza* e non è limitato a priori. In algebra relazionale quindi **non** è possibile scrivere un'unica espressione che calcoli la chiusura transitiva di una qualunque relazione binaria.

3.21 Datalog

Linguaggio di programmazione logica per basi di dati, derivato dal Prolog. Usa due tipi di predicati:

- **estensionali**: le relazioni della base di dati;
- **intensionali**: corrispondono alle viste, e sono definiti tramite **regole**.

Sintassi

$head \leftarrow body$

- *head* (testa) è un predicato atomico intensionale;
- *body* (corpo) è una lista (congiunzione) di predicati atomici;
- le interrogazioni sono predicati atomici preceduti da ?.

Esempi

- Impiegati di 30 anni (basta un predicato estensionale con una costante):

? Employee(Number : m , Name : n , Age : 30, Salary : s)

- Impiegati che guadagnano più di 40 (serve un predicato intensionale):

RichE(Number : m , Name : n , Age : e , Salary : s) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s), $s > 40$
? RichE(Number : m , Name : n , Age : e , Salary : s)

- Numero, nome ed età di tutti gli impiegati (proiezione):

PublicInfos(Number : m , Name : n , Age : e) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s)
? PublicInfos(Number : m , Name : n , Age : e)

- Capi degli impiegati che guadagnano più di 40 (join):

ChiefsOfRichE(Chief : c) $\leftarrow$
Employee(Number : m , Name : n , Age : e , Salary : s), $s > 40$,
Supervisor(Chief : c , Employee : m)
? ChiefsOfRichE(Chief : c)

- Capi i cui impiegati guadagnano *tutti* più di 40 (serve la **negazione**):

ChiefsOfNoRichE(Chief : c) $\leftarrow$
Supervisor(Chief : c , Employee : m),
Employee(Number : m , Name : n , Age : e , Salary : s), $s \leq 40$
ChiefsOfOnlyRichE(Number : c , Name : n) $\leftarrow$
Employee(Number : c , Name : n , Age : e , Salary : s),
Supervisor(Chief : c , Employee : m),
not ChiefsOfNoRichE(Chief : c)
? ChiefsOfOnlyRichE(Number : c , Name : n)

- Per ogni impiegato, tutti i suoi superiori (serve la **ricorsione**: è la chiusura transitiva):

HighGrade(Employee : i , SuperChief : c) $\leftarrow$ Supervisor(Employee : i , Chief : c)
HighGrade(Employee : i , SuperChief : c) $\leftarrow$
Supervisor(Employee : i , Chief : c'), HighGrade(Employee : c' , SuperChief : c)

Semantica e potere espressivo la definizione della semantica delle interrogazioni ricorsive è delicata, soprattutto in presenza di negazione.

- Datalog *non ricorsivo senza negazione* equivale al calcolo senza negazione e senza quantificatore universale;
- Datalog *non ricorsivo con negazione* equivale a calcolo e algebra;
- Datalog *ricorsivo senza negazione* e calcolo sono *incomparabili*;
- Datalog *ricorsivo con negazione* è più espressivo di calcolo e algebra.

4 SQL di base

4.1 SQL

In origine acronimo di *Structured Query Language*, oggi è un nome proprio. È un linguaggio con molte funzionalità: implementa sia il **DDL** sia il **DML**. Esiste uno standard ISO, ma ogni DBMS ha la propria grammatica (dialetto).

4.2 Storia

- il predecessore è **SEQUEL** (1974);
- prime implementazioni: SQL/DS e Oracle (1981);
- standard “de facto” dal 1983;
- molti aggiornamenti (1986, 1989, 1992, 1999, 2003, 2006, 2008, ...), ma i DBMS mantengono comunque la propria grammatica.

Nome non ufficiale	Nome ufficiale	Caratteristiche
SQL-Base	SQL-86	primo standard: quasi tutte le clausole per le interrogazioni, supporto limitato a creazione e modifica di schemi e dati
	SQL-89	integrità referenziale
SQL-2	SQL-92	modello relazionale, nuove funzioni (COALESCE, NULLIF, CASE); tre livelli: entry, intermediate, full; in gran parte retrocompatibile
SQL-3	SQL:1999	modello object-relational, trigger, funzioni esterne
	SQL:2003	estende il supporto a oggetti; query in Java e su dati semistrutturati (SQL/XML); rimosse parole chiave in disuso
	SQL:2006	supporto esteso a XML (XQuery)
	SQL:2008	piccole modifiche di sintassi (es. trigger INSTEAD OF)

4.3 Definizione dei dati (DDL)

CREATE DATABASE ogni database creato contiene tabelle, viste, trigger e altro. **CREATE DATABASE db_name** (in SQLite `sqlite3_open_v2(db_name)`, in Mimer **CREATE DATABANK**).

CREATE SCHEMA uno schema SQL ha un nome e descrive gli elementi che gli appartengono (tabelle, tipi, vincoli, viste, domini, ...). Appartiene all'utente che ha eseguito il comando; con **AUTHORIZATION** si indica un utente proprietario diverso:

```
CREATE SCHEMA schema_name AUTHORIZATION 'user_name'
```

CREATE TABLE definisce una nuova relazione e ne crea l'istanza (vuota); specifica gli attributi (con i loro tipi) e i vincoli iniziali.

```
CREATE TABLE Employee(
  Number CHAR(6) PRIMARY KEY,
  Name CHAR(20) NOT NULL,
  Surname CHAR(20) NOT NULL,
  Dept CHAR(15),
  Wage NUMERIC(9) DEFAULT 0,
  FOREIGN KEY(Dept) REFERENCES Department(Dept),
  UNIQUE (Surname, Name)
)
```

4.4 Tipi di dato

I tipi degli attributi corrispondono ai *domini* del modello relazionale. Si distinguono tipi **base** (predefiniti) e tipi **custom** (“domini”, semplici e riutilizzabili).

Tipi base

- stringhe di caratteri, a lunghezza fissa (**CHAR(n)**) o variabile (**VARCHAR(n)**);
- numerici: interi e vari tipi in virgola mobile;
- DATE, TIME, INTERVAL;
- introdotti con SQL-3: BOOLEAN, BLOB e CLOB (*binary/character large object*, grandi collezioni di dati, testuali o no).

CREATE DOMAIN definisce un tipo custom, riusabile nelle relazioni, con vincoli e valore di default:

```
CREATE DOMAIN Grade
  AS SMALLINT DEFAULT NULL
  CHECK ( value >= 18 AND value <= 30 )
```

4.5 Vincoli intrarelazionali

- NOT NULL;
- UNIQUE: definisce una chiave (superchiave);
- PRIMARY KEY: una sola per tabella, implica NOT NULL (DB2 ha un comportamento non standard);
- CHECK: vedi SQL avanzato.

UNIQUE e PRIMARY KEY si possono scrivere *accanto all'attributo* (se la chiave è un solo attributo) o come *elemento a sé*:

```
Number CHAR(6) PRIMARY KEY
-- equivale a
Number CHAR(6),
PRIMARY KEY (Number)
```

Attenzione le due definizioni seguenti **non** sono equivalenti:

```
Name      CHAR(20) NOT NULL,
Surname    CHAR(20) NOT NULL,
UNIQUE (Surname, Name)          -- la COPPIA è chiave
```

```
Name      CHAR(20) NOT NULL UNIQUE,
Surname    CHAR(20) NOT NULL UNIQUE -- OGNI attributo è chiave (più restrittivo)
```

4.6 Vincoli interrelazionali

REFERENCES e FOREIGN KEY definiscono vincoli di **integrità referenziale**, su un singolo attributo o su più attributi. Si possono definire *azioni* da eseguire quando il vincolo viene violato.

Esempio Offences(Code, Day, Officer, State, Number) con Officer che riferisce Officer(Number, Surname, Name) e la coppia (State, Number) che riferisce Car(State, Number, Surname, Name):

```
CREATE TABLE Offences(
  Code      CHAR(6) NOT NULL PRIMARY KEY,
  Day       DATE NOT NULL,
  Officer   INTEGER NOT NULL REFERENCES Officer(Number),
  State     CHAR(2),
  Number    CHAR(6),
  FOREIGN KEY(State, Number) REFERENCES Car(State, Number)
)
```

Azioni referenziali (referential triggered actions) dopo ogni vincolo referenziale si può specificare l'azione da compiere quando un'operazione sulla tabella riferita violerebbe il vincolo:

```
ON < DELETE | UPDATE >
  < CASCADE | SET NULL | SET DEFAULT | NO ACTION >
```

CASCADE

su *delete*: si cancellano le tuple che riferiscono; su *update*: il valore della chiave esterna viene aggiornato al nuovo valore.

SET NULL

l'attributo che riferisce viene posto a NULL.

SET DEFAULT

l'attributo che riferisce assume il valore di default.

NO ACTION

la cancellazione/modifica non è permessa (default).

4.7 Modifica degli schemi

Comandi: ALTER DOMAIN, ALTER TABLE, DROP DOMAIN, DROP TABLE.

ALTER DOMAIN modifica un dominio già definito; si usa con SET DEFAULT, DROP DEFAULT, ADD CONSTRAINT, DROP CONSTRAINT.

```
ALTER DOMAIN Grade SET DEFAULT 30    -- default applicato solo d'ora in poi,  
                                     -- ai valori mancanti inseriti
```

```
ALTER DOMAIN Grade DROP DEFAULT
```

```
ALTER DOMAIN Grade ADD CONSTRAINT isValid CHECK (value >= 18 AND value <= 30)
```

```
ALTER DOMAIN Grade DROP CONSTRAINT isValid
```

ALTER TABLE modifica una tabella già definita; si usa con ALTER COLUMN, ADD COLUMN, DROP COLUMN, ADD CONSTRAINT, DROP CONSTRAINT.

```
ALTER TABLE Employee ALTER COLUMN Number SET NOT NULL
```

```
ALTER TABLE Employee ADD COLUMN Level CHARACTER(10)
```

```
ALTER TABLE Employee DROP COLUMN Level RESTRICT -- solo se non contiene valori
```

```
ALTER TABLE Employee DROP COLUMN Level CASCADE -- la rimuove con i suoi valori
```

```
ALTER TABLE Employee ADD CONSTRAINT validNum CHECK (char_length(Number) = 10)
```

```
ALTER TABLE Employee DROP CONSTRAINT validNum
```

DROP DOMAIN / DROP TABLE DROP DOMAIN Grade rimuove un tipo definito dall'utente; DROP TABLE Offences rimuove l'intera tabella: schema e dati.

4.8 Indici

Migliorano i tempi di risposta delle interrogazioni (efficienza). Sono definiti a livello *fisico*, non logico; un tempo erano anche l'unico modo per definire chiavi.

```
CREATE INDEX idx_Surname ON Officer (Surname)
```

DDL in pratica in molti sistemi e progetti, per definire lo schema si usano strumenti (spesso grafici) al posto delle istruzioni SQL.

4.8.1 Strutture fisiche di accesso

I dati stanno su memoria secondaria, organizzata in **blocchi** (pagine): il costo di un'operazione si misura nel *numero di blocchi letti o scritti*, perché l'accesso al disco domina il tempo di calcolo. Un file di B blocchi può essere organizzato come:

Heap (seriale)

record inseriti in fondo, senza ordine. Inserimento economico, ricerca per scansione.

Sequenziale ordinato

record ordinati su un campo (*pseudochiave*); permette ricerca binaria e scansioni ordinate, ma gli inserimenti richiedono spazio libero o blocchi di overflow.

Hash

la posizione del record si calcola con una funzione hash sulla chiave: accesso diretto per uguaglianza.

4.8.2 Classificazione degli indici

Un **indice** è una struttura ausiliaria (come l'indice analitico di un libro) che associa ai valori della *chiave di ricerca* i puntatori ai record. Classificazioni:

Primario / secondario

primario se è definito sul campo che determina l'ordinamento del file (spesso la chiave primaria; es. file di studenti ordinato per matricola, indice su matricola); *secondario* se definito su un altro campo (es. indice su cognome). Gli indici secondari sono sempre densi.

Denso / sparso

denso se ha un'entrata per ogni valore della chiave presente nel file; *sparso* se ne ha meno (tipicamente una per blocco): possibile solo se il file è ordinato sulla chiave dell'indice.

Clustered / unclustered

clustered se l'ordine fisico dei record segue (o quasi) quello delle entrate dell'indice: rende efficienti le ricerche su intervalli (es. `data BETWEEN ...`).

Monolivello / multilivello

se l'indice stesso è grande, si costruisce un indice sull'indice: si ottiene una struttura gerarchica ad albero.

4.8.3 Alberi B e B+

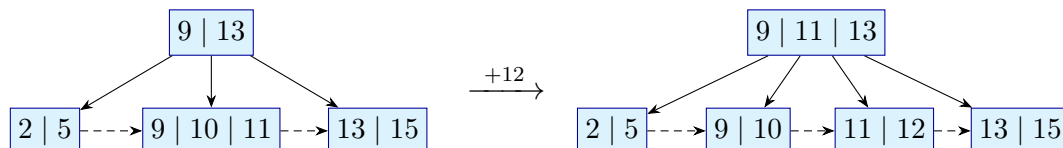
Gli indici dinamici dei DBMS sono **alberi bilanciati** di *fan-out* elevato (ogni nodo è un blocco con molte chiavi), quindi bassi: 3–4 livelli bastano per milioni di record.

- Ogni nodo contiene fino a $F - 1$ chiavi ordinate e F puntatori; i nodi (radice esclusa) devono essere pieni almeno a metà.
- Tutte le foglie sono allo stesso livello (*bilanciamento*).
- **B-tree**: i nodi intermedi puntano anche ai record dei dati.
- **B+tree**: tutte le chiavi compaiono nelle foglie, che puntano ai dati e sono *collegate in lista*; i nodi intermedi contengono solo chiavi separatrici. La lista delle foglie rende efficienti le ricerche su intervalli.

Ricerca dalla radice si scende scegliendo il puntatore tra le due chiavi che racchiudono il valore cercato ($k_i \leq v < k_{i+1}$), fino alla foglia. Costo: altezza dell'albero ($\approx \log_F N$ accessi).

Inserimento si cerca la foglia giusta e si inserisce la chiave in ordine. Se la foglia è piena si fa uno **split**: le chiavi si dividono in due foglie, e la prima chiave della nuova foglia viene *copiata* nel padre (che a sua volta può dividersi; lo split di un nodo intermedio *sposta* la chiave mediana verso l'alto). Se si divide la radice, l'albero cresce di un livello.

Esempio (nodi da 3 chiavi): inserimento di 12. La foglia $[9, 10, 11]$ è piena: si divide in $[9, 10]$ e $[11, 12]$ e la chiave 11 sale nella radice.



Cancellazione si elimina la chiave dalla foglia. Se la foglia va in **underflow** (meno di metà piena):

- se un nodo fratello ha chiavi in eccesso si fa una **ridistribuzione**: il fratello cede una chiave e si aggiorna la chiave separatrice nel padre;
- altrimenti si fa un **merge** (fusione) con il fratello e si elimina la separatrice dal padre, che può andare a sua volta in underflow.

Esempio: radice [8 | 13], foglie [2, 4, 5], [8, 11], [13, 15]; tolgo 8: la foglia [11] va in underflow, la sorella [2, 4, 5] le cede 5, le foglie diventano [2, 4] e [5, 11] e la separatrice nella radice diventa 5.

4.8.4 Strutture hash

Una funzione hash $h(k)$ trasforma la chiave nel numero di un *bucket* (uno o più blocchi). Ottime per ricerche per *uguaglianza* (circa un accesso), inutili per ricerche su intervalli.

Hashing statico

numero di bucket fisso: quando un bucket è pieno i record in più vanno in **catene di overflow**, che allungano la ricerca se il file cresce.

Hashing dinamico / lineare

il numero di bucket cresce col file. Nell'hashing lineare i bucket si dividono *a turno*, seguendo un puntatore di split, e non necessariamente quello pieno: anche qui servono pagine di overflow temporanee.

4.8.5 Costo delle operazioni

Numero medio di blocchi letti, file di B blocchi:

Operazione	Organizzazione	Costo
Uguaglianza sulla chiave	heap	$0.5 B$ (ci si ferma al record trovato)
Ricerca su campo non chiave	heap	B (scansione completa)
Uguaglianza sul campo di ordinamento	file ordinato	$\log_2 B$ (ricerca binaria)
Uguaglianza su un altro campo (anche UNIQUE)	file ordinato	$0.5 B$, come un heap
Uguaglianza	hash (senza overflow)	≈ 1
Uguaglianza o intervallo	B+tree	altezza + blocchi letti

4.9 Interrogazioni: SELECT

Operazioni sui dati: **interrogazione** (SELECT) e **modifica** (INSERT, DELETE, UPDATE).

```
SELECT <ListaAttributi>      -- target list
FROM <ListaTabelle>         -- clausola FROM
[ WHERE <Condizione> ]      -- clausola WHERE
```

Come leggere una SELECT l'ordine di valutazione non è quello di scrittura:

```
3 SELECT Number, Name
1 FROM Officer
2 WHERE Surname = 'Jones'
```

(1) si prende la tabella `Officer`, (2) si tengono gli ufficiali con cognome `'Jones'`, (3) di ciascuno si mostrano `Number` e `Name`.

Database di esempio usato nel seguito:

People							
Name	Age	Income	Motherhood		Fatherhood		
Jim	27	21	Mother	Child	Father	Child	
James	25	15					
Alice	55	42	Abby	Alice	Steve	Frank	
Jesse	50	35	Abby	Louis	Louis	Olga	
Phil	26	30	Jesse	Olga	Louis	Phil	
Louis	50	40	Jesse	Phil	Frank	Jim	
Frank	60	20	Alice	Jim	Frank	James	
Olga	30	41	Alice	James			
Steve	85	35					
Abby	75	87					

4.10 Selezione e proiezione

Selezione e proiezione nome e reddito delle persone con meno di 30 anni, $\pi_{\text{Name,Income}}(\sigma_{\text{Age}<30}(\text{People}))$:

```
SELECT Name, Income
FROM People
WHERE Age < 30
```

Risultato: (Jim, 21), (James, 15), (Phil, 30).

Solo selezione $\sigma_{\text{Age}<30}(\text{People})$: si usa `SELECT *` (tutti gli attributi).

```
SELECT * FROM People WHERE Age < 30
```

Solo proiezione $\pi_{\text{Name,Income}}(\text{People})$: si omette il `WHERE`.

```
SELECT Name, Income FROM People
```

Ridenominazione con `AS` si rinominano attributi (nella target list) e tabelle (nel `FROM`, *alias* o variabili di tupla):

```
SELECT p.Name AS GivenName, p.Income AS Revenue
FROM People AS p
WHERE p.Age < 30
```

Abbreviazioni data $R(A, B)$, `SELECT * FROM R` equivale a

```
SELECT X.A AS A, X.B AS B
FROM R X
WHERE true
```

Condizioni composte con AND, OR, NOT:

```
SELECT * FROM People
WHERE Income > 25 AND (Age < 30 OR Age > 60)
```

Risultato: Phil (26, 30) e Steve (85, 35).

Predicato LIKE confronto con pattern: `_` indica un carattere qualsiasi, `%` una sequenza (anche vuota) di caratteri. Persone con nome che inizia per 'J' e ha 'm' come terza lettera:

```
SELECT * FROM People WHERE Name LIKE 'J_m%'
```

Risultato: Jim, James.

Valori nulli si usano `IS NULL` / `IS NOT NULL`. Impiegati con più di 40 anni o età sconosciuta, $\sigma_{(Age>40) \vee (Age \text{ IS NULL})}(\text{Employee})$:

```
SELECT * FROM Employee WHERE Age > 40 OR Age IS NULL
```

Duplicati e DISTINCT a differenza dell'algebra, SQL **non** elimina i duplicati (le tabelle sono *multiinsiemi*). Per eliminarli si usa `DISTINCT`:

```
SELECT Surname, Agency FROM Employee          -- (Neri,Naples), (Neri,Milan),
                                                -- (Rossi,Rome), (Rossi,Rome)
SELECT DISTINCT Surname, Agency FROM Employee -- (Rossi,Rome) una volta sola
```

4.11 SQL e algebra relazionale

Con una sola relazione nel `FROM` si esprimono selezione, proiezione e ridenominazione; con più relazioni si ottengono **join** (e prodotti cartesiani). Date $R_1(A_1, A_2)$ e $R_2(A_3, A_4)$:

```
SELECT DISTINCT R1.A1, R2.A4
FROM R1, R2
WHERE R1.A2 = R2.A3
```

corrisponde a $\pi_{A_1, A_4}(\sigma_{A_2=A_3}(R_1 \bowtie R_2))$: il `FROM` fa il prodotto cartesiano, il `WHERE` la selezione, la `SELECT` la proiezione.

Alias la ridenominazione serve nel prodotto cartesiano (per usare più copie della stessa tabella) e nella target list:

```
SELECT DISTINCT X.A1 AS B1, Y.A4 AS B2
FROM R1 X, R2 Y, R1 Z
WHERE X.A2 = Y.A3 AND Y.A4 = Z.A1
```

$$\rho_{B_1, B_2 \leftarrow A_1, A_4}(\pi_{A_1, A_4}(\sigma_{A_2=A_3 \wedge A_4=C_1}(R_1 \bowtie R_2 \bowtie \rho_{C_1, C_2 \leftarrow A_1, A_2}(R_1))))$$

Valutazione SQL è dichiarativo: la semantica è data “per esempi”. I DBMS usano *piani di esecuzione* efficienti (selezioni il prima possibile, join al posto dei prodotti cartesiani). Non serve quindi scrivere query efficienti: è più importante che siano *comprensibili* (per evitare errori).

4.12 Join

Esempio 1 padri delle persone che guadagnano più di 20, $\pi_{\text{Father}}(\text{Fatherhood} \bowtie_{\text{Child=Name}} \sigma_{\text{Income}>20}(\text{People}))$:

```
SELECT DISTINCT Father
FROM People, Fatherhood
WHERE Child = Name AND Income > 20
```

Esempio 2 nome e reddito delle persone che guadagnano più del padre, con il reddito del padre (serve People due volte):

```
SELECT c.Name AS Name, c.Income AS Income, f.Income AS fatherIncome
FROM People f, Fatherhood, People c
WHERE f.Name = Father AND Child = c.Name
      AND c.Income > f.Income
```

In algebra: $\pi_{\text{Name,Income,IF}}(\sigma_{\text{Income}>\text{IF}}(\rho_{\text{NF,AF,IF}\leftarrow\text{Name,Age,Income}}(\text{People}) \bowtie_{\text{NF=Father}} (\text{Fatherhood} \bowtie_{\text{Child=Name}} \text{People})))$.

Espressioni nella target list

```
SELECT Income/2 AS halvedIncome FROM People WHERE Name = 'Louis' -- 20
```

Join esplicito madre e padre di ogni persona.

```
-- join implicito
SELECT f.child, father, mother
FROM motherhood m, fatherhood f
WHERE m.child = f.child

-- join esplicito
SELECT mother, fatherhood.child, father
FROM motherhood JOIN fatherhood ON fatherhood.child = motherhood.child
```

Sintassi:

```
SELECT ...
FROM LeftTable { ... JOIN RightTable ON joincondition }, ...
[ WHERE otherPredicate ]
```

L'esempio 2 con join espliciti:

```
SELECT c.name, c.income, f.income
FROM (people f JOIN fatherhood ON f.name = father)
      JOIN people c ON child = c.name
WHERE c.income > f.income
```

Join naturale $\pi_{\text{Child,Father,Mother}}(\text{Fatherhood} \bowtie_{\text{Child=Name}} \rho_{\text{Name}\leftarrow\text{Child}}(\text{Motherhood}))$, cioè $\text{Fatherhood} \bowtie \text{Motherhood}$:

```
SELECT mother, child, father
FROM Motherhood NATURAL JOIN Fatherhood
```

4.13 Join esterno

Con i join visti finora (*inner join*) le tuple senza corrispondente nell'altra tabella vengono scartate. Per non perdere informazione si usa **LEFT/RIGHT/FULL OUTER JOIN**; per left e right la parola **OUTER** si può omettere (sono “esterni” per definizione).

Left join padre e, se nota, madre di ogni persona:

```
SELECT Fatherhood.child, father, mother
FROM Fatherhood LEFT [OUTER] JOIN Motherhood
    ON Fatherhood.child = Motherhood.child
```

Risultato: (Frank, Steve, NULL), (Olga, Louis, Jesse), (Phil, Louis, Jesse), (Jim, Frank, Alice), (James, Frank, Alice).

Full join restituisce anche le tuple escluse da entrambi gli operandi:

```
SELECT Fatherhood.child, father, mother
FROM Motherhood FULL OUTER JOIN Fatherhood
    ON Motherhood.child = Fatherhood.child
```

Oltre alle 5 righe del left join compaiono (NULL, NULL, Abby) due volte (Alice e Louis non hanno padre in Fatherhood).

Riepilogo

A INNER JOIN B ON A.Key = B.Key	$A \cap B$
A LEFT JOIN B ON A.Key = B.Key	tutta A
A RIGHT JOIN B ON A.Key = B.Key	tutta B
A FULL OUTER JOIN B ON A.Key = B.Key	$A \cup B$
... LEFT JOIN ... WHERE B.Key IS NULL	$A \setminus B$
... RIGHT JOIN ... WHERE A.Key IS NULL	$B \setminus A$
... FULL JOIN ... WHERE A.Key IS NULL OR B.Key IS NULL	$A \triangle B$ (differenza simmetrica)

4.14 Ordinamento

```
SELECT name, income
FROM People
WHERE age < 30
ORDER BY name ASC
```

ASC ordine crescente, DESC decrescente. Senza **ORDER BY** l'ordine delle righe non è definito. (Le slide dicono che il default è decrescente, ma negli esempi **ORDER BY name** dà James, Jim, Phil: il default standard è **ASC**.)

4.15 Unione, intersezione, differenza

La **SELECT** da sola non esprime l'unione: serve un costrutto apposito.

```
SELECT ... UNION [ALL] SELECT ...
```

Il risultato è senza duplicati, salvo con **ALL** (unione di *multiinsiemi*).

```
SELECT child FROM Motherhood
UNION          -- Alice, Louis, Olga, Frank, Phil
SELECT child FROM Fatherhood
```

```
SELECT child FROM Motherhood
UNION ALL      -- Olga compare due volte
SELECT child FROM Fatherhood
```

Notazione posizionale se gli schemi hanno nomi diversi non si rinomina: si assumono i nomi del **primo operando** e gli attributi si abbinano per posizione.

```
SELECT Father, Child FROM Fatherhood
UNION
SELECT Mother, Child FROM Motherhood  -- schema risultante: (Father, Child)
```

Attenzione: `SELECT father, child ... UNION SELECT child, mother ...` ha schema `(father, child)` ma mette i figli nella colonna dei padri!

Differenza e intersezione

```
SELECT Name FROM Employee
EXCEPT          -- INTERSECT per l'intersezione
SELECT Surname AS Name FROM Employee
```

L'intersezione si può esprimere anche con un join:

```
SELECT E.Name FROM Employee E, Employee F WHERE E.Name = F.Surname
```

La differenza si esprime anche con interrogazioni annidate.

4.16 Interrogazioni annidate

Nel `WHERE` i predicati possono confrontare uno (o più) attributi con il risultato di una *sottointerrogazione*, o usare il quantificatore esistenziale.

Esempio nome e reddito del padre di Frank:

```
SELECT Name, Income          -- prodotto cartesiano + equi-join
FROM People, Fatherhood
WHERE Name = Father AND Child = 'Frank'
```

```
SELECT Name, Income          -- annidata: vera se Name è uguale
FROM People                  -- al risultato (una sola tupla)
WHERE Name = (SELECT Father FROM Fatherhood WHERE Child = 'Frank')
```

Discussione le query annidate sono “meno dichiarative”, ma a volte più leggibili (meno variabili). Si possono combinare con quelle non annidate. Le sottointerrogazioni non possono contenere operazioni insiemistiche (“l’unione si fa nella query esterna”): limite poco significativo.

Semantica l’interrogazione interna viene eseguita **una volta per ogni tupla** dell’interrogazione esterna.

ANY, ALL, IN

Attributo op ANY (Expr)

la tupla è selezionata se soddisfa il confronto op ($=, <, >, \geq, \dots$) con *almeno una* tupla di Expr.

Attributo op ALL (Expr)

... con *tutte* le tuple di Expr.

Attributo IN (Expr)

il valore è contenuto nel risultato di Expr (equivale a $=$ ANY).

Si negano antepoendo NOT.

Esempio nome e reddito dei padri con almeno un figlio che guadagna più di 20:

```
SELECT DISTINCT F.Name, F.Income
FROM People F, Fatherhood, People C
WHERE F.Name = Father AND Child = C.Name AND C.Income > 20
```

Senza DISTINCT (non facendo join, i padri non si ripetono per ogni figlio):

```
SELECT Name, Income
FROM People
WHERE Name IN (SELECT Father
                FROM Fatherhood
                WHERE Child = ANY (SELECT Name
                                   FROM People
                                   WHERE Income > 20))
```

-- oppure

```
WHERE Name IN (SELECT Father FROM Fatherhood, People
                WHERE Child = Name AND Income > 20)
```

Se però si vuole anche il reddito del figlio, la versione annidata **non** funziona: nella query esterna i dati del figlio non sono visibili. Serve la versione con join (SELECT DISTINCT F.Name, F.Income, C.Income ...).

Regole di visibilità

- non ci si può riferire a variabili definite in blocchi *più interni*;
- se il nome di una variabile è omissso, si assume quella dichiarata “più vicina”;
- ci si può riferire a variabili dei blocchi esterni.

Esempio **errato**: D1.City non è visibile nella seconda sottointerrogazione, perché D1 è definita in un blocco parallelo:

```
SELECT * FROM Employee
WHERE Dept IN (SELECT Name FROM Department D1
               WHERE Name = 'Production') OR
              Dept IN (SELECT Name FROM Department D2
                       WHERE D2.City = D1.City)
```

4.17 Quantificatore esistenziale

EXISTS (Expr) è vero se Expr restituisce almeno una tupla.

Persone con almeno un figlio (Name è preso dalla relazione del blocco esterno):

```
SELECT * FROM People
WHERE EXISTS (SELECT * FROM Fatherhood WHERE Father = Name) OR
            EXISTS (SELECT * FROM Motherhood WHERE Mother = Name)
```

Padri i cui figli guadagnano tutti più di 20 (NOT EXISTS esprime il “per ogni”):

```
SELECT DISTINCT Father
FROM Fatherhood Z
WHERE NOT EXISTS (SELECT *
                  FROM Fatherhood W, People
                  WHERE W.Father = Z.Father
                        AND W.Child = Name
                        AND Income <= 20)
```

Errore di scope nella versione seguente Child, senza riferimento a tabella, si riferisce implicitamente al FROM più vicino, ma People non ha Child; va scritto F.Child:

```
SELECT DISTINCT F.Father
FROM Fatherhood AS F
WHERE NOT EXISTS (SELECT * FROM People
                  WHERE F.Child = Name AND Income <= 20)
```

Differenza con query annidate

```
SELECT Name FROM Employee
EXCEPT
SELECT Surname AS Name FROM Employee
-- equivale a
SELECT Name FROM Employee I
WHERE NOT EXISTS (SELECT * FROM Employee WHERE Surname = I.Name)
```

4.18 Funzioni aggregate

Nella target list si possono usare espressioni che calcolano un valore a partire da un *insieme* di tuple: COUNT, MIN, MAX, AVG, SUM.

```
Aggr([DISTINCT] *)
Aggr([DISTINCT] Attributo)
```

COUNT numero di figli di Frank (la funzione si applica alle tuple del risultato SELECT * FROM Fatherhood WHERE father = 'Frank'):

```
SELECT count(*) AS NumFrankChildren
FROM Fatherhood
WHERE father = 'Frank' -- 2
```

- COUNT(*) conta le righe;

- COUNT(attr) conta i valori **non nulli** di attr;
- COUNT(DISTINCT attr) conta i valori distinti non nulli.

Con redditi (30, NULL, 36, 36): COUNT(*) = 4, COUNT(income) = 3, COUNT(DISTINCT income) = 2.

SUM, AVG, MAX, MIN ignorano i NULL. Media dei redditi dei figli di Frank:

```
SELECT AVG(income)
FROM People JOIN Fatherhood ON name = child
WHERE father = 'Frank'
```

Con redditi (21, NULL, 21, 35) AVG(income) = $77/3 \approx 25,6$ (il NULL non conta nemmeno al denominatore).

Aggregate e target list query errata:

```
SELECT name, MAX(income) FROM People
```

Di chi è il nome? La target list deve essere “omogenea”: o tutte aggregate, o (con raggruppamento) attributi di raggruppamento. Corretta: SELECT MIN(age), MAX(income) FROM People. Per le persone con il reddito massimo si usa una query annidata:

```
SELECT * FROM People
WHERE income = (SELECT MAX(income) FROM People)
```

4.19 Raggruppamento

Le funzioni aggregate si possono applicare a *gruppi* di tuple con GROUP BY ListaAttributi. Numero di figli di ogni padre:

```
SELECT Father, COUNT(*) AS NumberOfChildren
FROM Fatherhood
GROUP BY Father           -- (Steve,1), (Louis,2), (Frank,2)
```

Semantica (1) si esegue la query senza aggregati né raggruppamento (SELECT * FROM Fatherhood); (2) si raggruppano le tuple con stessi valori degli attributi di GROUP BY e si applica l'aggregato a ogni gruppo.

Raggruppamento e target list nella target list possono comparire solo attributi del GROUP BY e funzioni aggregate. Errata:

```
SELECT father, AVG(f.income), c.income
FROM People c JOIN Fatherhood ON child = c.name
      JOIN People f ON father = f.name
GROUP BY father           -- c.income non è unico nel gruppo (più figli)
```

Corretta: GROUP BY father, c.income.

Condizioni sui gruppi: HAVING padri i cui figli hanno reddito medio maggiore di 25:

```
SELECT father, AVG(f.income)
FROM People f JOIN Fatherhood ON child = name
GROUP BY father
HAVING AVG(f.income) > 25
```

WHERE vs HAVING WHERE filtra le *tuple* prima del raggruppamento, HAVING filtra i *gruppi* (condizioni su aggregati). Padri i cui figli sotto i 30 anni hanno reddito medio maggiore di 20:

```
SELECT father, AVG(f.income)
FROM People f JOIN Fatherhood ON child = name
WHERE age < 30
GROUP BY father
HAVING AVG(f.income) > 20
```

Raggruppamento e NULL il NULL forma un gruppo a sé. Con $R(A, B) = \{(1, 11), (2, 11), (3, \text{NULL}), (4, 1)\}$

- SELECT B, COUNT(*) FROM R GROUP BY B $\rightarrow (11, 2), (\text{NULL}, 2)$;
- SELECT A, COUNT(*) FROM R GROUP BY A $\rightarrow 1$ per ogni A;
- SELECT A, COUNT(B) FROM R GROUP BY A $\rightarrow (1,1), (2,1), (3,0), (4,0)$.

Sintassi completa della SELECT

```
SELECT ListaAttributi + Espressioni
FROM ListaTabelle + Join
[ WHERE Condizione ]
[ GROUP BY ListaAttributiRaggruppamento ]
[ HAVING CondizioneAggregata ]
[ ORDER BY AttributiOrdinamento ]
```

4.20 Modifica dei dati

INSERT, DELETE, UPDATE agiscono su una o più tuple di una tabella, in base a un predicato che può coinvolgere altre relazioni.

INSERT

```
INSERT INTO Tabella [(ListaAttributi)] VALUES (Valori)
INSERT INTO Tabella [(ListaAttributi)] SELECT ...
```

```
INSERT INTO People VALUES ('John', 25, 52)
INSERT INTO People(Name, Age, Income) VALUES ('Jack', 25, 52)
INSERT INTO People(Name, Income) VALUES ('Robert', 55)
INSERT INTO People(Name)
  SELECT Father FROM Fatherhood
  WHERE Father NOT IN (SELECT Name FROM People)
```

- l'ordine di attributi e valori conta; le due liste devono avere lo stesso numero di elementi;
- se la lista attributi è omessa si intendono tutti, nell'ordine dello schema;
- gli attributi non elencati ricevono NULL o il valore di default.

DELETE

```
DELETE FROM Tabella [ WHERE Condizione ]
```

```
DELETE FROM People WHERE Age < 35
```

```
DELETE FROM Fatherhood WHERE Child NOT IN (SELECT Name FROM People)
```

```
DELETE FROM Fatherhood          -- svuota la tabella
```

Cancella le tuple che soddisfano la condizione; può causare la cancellazione di tuple di altre tabelle (vincoli con **CASCADE**). Senza condizione si intende **WHERE TRUE**.

UPDATE

```
UPDATE NomeTabella
```

```
SET Attributo = < Expr | SELECT ... | NULL | DEFAULT >  
[ WHERE Condizione ]
```

Esempi su `People = {(Jim, 27, 30), (James, 25, 15), (Bob, 55, 36)}`:

```
UPDATE People SET Income = 45 WHERE Name = 'Bob'          -- Bob: 45
```

```
UPDATE People SET Income = Income * 1.1 WHERE Age < 30    -- Jim 33, James 16,5
```

```
UPDATE People SET Income = (SELECT Income FROM People WHERE Name = 'Jim')  
WHERE Name = 'Bob'                                         -- Bob: 30
```

```
UPDATE People SET Income = NULL WHERE Age < 30            -- Jim, James: NULL
```

```
UPDATE People SET Income = DEFAULT WHERE Age < 30         -- Jim, James: 0
```

(l'ultimo assumendo **DEFAULT 0** per `Income` nella **CREATE TABLE**).

5 SQL avanzato

5.1 Vincoli CHECK

CHECK (Predicato) specifica vincoli sulle tuple (ed eventualmente vincoli più complessi, non supportati da tutte le implementazioni).

```
CREATE TABLE Employee (  
    Number      CHARACTER(6),  
    Surname     CHARACTER(20),  
    Name        CHARACTER(20),  
    Sex         CHARACTER NOT NULL CHECK (Sex IN ('M','F')),  
    Salary      INTEGER CHECK (Salary >= 0),  
    Withholding INTEGER,  
    Net         INTEGER,  
    Supervisor  CHARACTER(6),  
    CHECK (Net = Salary - Withholding),  
    CHECK (Salary <= (SELECT Salary FROM Employee J  
                        WHERE Supervisor = J.Number))  
)
```

Una **SELECT** dentro un **CHECK** (ultimo vincolo: nessuno guadagna più del proprio capo) non è supportata ovunque. Con i primi vincoli:

- `INSERT INTO Employee VALUES (1,'Doe','John',,,100,20,80)` → rifiutata: Sex non è né 'M' né 'F';
- `(2,'Smith','Jimmy','M',100,10,80)` → rifiutata: $80 \neq 100 - 10$;
- `(3,'Rossini','Luca','M',70,20,50)` → accettata.

5.2 Asserzioni

Vincoli definiti a livello di **schema** (non legati a una tabella):

```
CREATE ASSERTION NomeAsserzione CHECK (Predicato)
```

```
CREATE ASSERTION AtLeastOneEmployee  
CHECK (1 <= (SELECT count(*) FROM Employee))
```

Anche qui la SELECT nel CHECK non è supportata da tutti.

5.3 Viste

```
CREATE VIEW NomeVista [ (ListaAttributi) ] AS SelectSQL  
[ WITH [ LOCAL | CASCADED ] CHECK OPTION ]
```

```
CREATE VIEW AdminEmployees (Name, Surname, Salary) AS  
SELECT Name, Surname, Salary  
FROM Employee  
WHERE Dept = 'Administration' AND Salary > 10
```

Interrogare una vista si fa come con qualsiasi relazione: `SELECT * FROM AdminEmployees` viene eseguita come la query che definisce la vista.

Aggiornamento delle viste di solito è permesso solo per viste definite su una *singola* relazione. Si può chiedere al DBMS di fare dei controlli con `CHECK OPTION`: l'aggiornamento è permesso solo se la tupla risultante appartiene ancora alla vista.

```
CREATE VIEW PoorAdminEmployees AS  
SELECT * FROM AdminEmployees  
WHERE Salary < 50  
WITH CHECK OPTION
```

```
UPDATE PoorAdminEmployees SET Salary = 60 WHERE Name = 'Ann' -- NON permessa
```

LOCAL e CASCADED per viste definite su altre viste:

LOCAL

il controllo si fa solo sull'ultimo livello (la vista stessa);

CASCADED

il controllo si fa su tutte le viste e relazioni sottostanti.

5.4 Viste per query complesse

Aggregati annidati numero medio di uffici per dipartimento. Errata, perché SQL non permette di annidare operatori aggregati:

```
SELECT avg(count(distinct Office)) FROM Employee GROUP BY Dept
```

Corretta, con una vista intermedia:

```
CREATE VIEW DeptOffices(NameDept, OffNo) AS  
SELECT Dept, count(distinct Office) FROM Employee GROUP BY Dept;
```

```
SELECT avg(OffNo) FROM DeptOffices
```

HAVING annidato dipartimento con la massima somma degli stipendi. Soluzione errata per alcuni sistemi (non tutti ammettono sottoquery con aggregati nella HAVING):

```
SELECT Dept FROM Employee
GROUP BY Dept
HAVING sum(Salary) >= ALL (SELECT sum(Salary) FROM Employee GROUP BY Dept)
```

Corretta:

```
CREATE VIEW BudgetSalary(Dep, TotalSalary) AS
  SELECT Dept, sum(Salary) FROM Employee GROUP BY Dept;

SELECT Dep FROM BudgetSalary
WHERE TotalSalary = (SELECT max(TotalSalary) FROM BudgetSalary)
```

5.5 Interrogazioni ricorsive

Per ogni persona trovare tutti gli antenati, data `Fatherhood(Father, Child)`. Serve la ricorrenza; in Datalog:

```
Ancestors(Ancessor :  $p$ , Descendant :  $f$ )  $\leftarrow$  Fatherhood(Father :  $p$ , Child :  $f$ )
Ancestors(Ancessor :  $a$ , Descendant :  $d$ )  $\leftarrow$  Fatherhood(Father :  $a$ , Child :  $f$ ), Ancestors(Ancessor :  $f$ , Descendant :  $d$ )
```

In SQL:1999 si usa `WITH RECURSIVE`, che definisce una vista costruita ricorsivamente:

```
WITH RECURSIVE Ancestors(Ancessor, Descendant) AS (
  SELECT Father, Child FROM Fatherhood          -- caso base
  UNION ALL
  SELECT Ancestor, Child                      -- passo ricorsivo
  FROM Ancestors, Fatherhood
  WHERE Descendant = Father
)
SELECT * FROM Ancestors
```

Esempio tutti i superiori di John Doe:

```
WITH RECURSIVE InCharge(No, Supervisor) AS (
  SELECT No, Supervisor FROM Employee
  UNION
  SELECT Employee.No, InCharge.Supervisor
  FROM Employee, InCharge
  WHERE Employee.Supervisor = InCharge.No
)
SELECT Name, Surname, InCharge.Supervisor
FROM Employee JOIN InCharge ON (Employee.No = InCharge.No)
WHERE Name = 'John' AND Surname = 'Doe'
```

5.6 Funzioni scalari

Funzioni a livello di tupla che restituiscono un singolo valore per tupla.

- **temporali**: `current_date` (data odierna), `extract(campo FROM espressione)` (estrae anno, mese, giorno, ora, ...);

- **stringhe:** `char_length` (lunghezza), `lower` (minuscolo);
- **conversione:** `CAST` converte un valore in un altro dominio;
- **condizionali:** `COALESCE`, `NULLIF`, `CASE`.

```
SELECT EXTRACT(YEAR FROM OrderDate) AS OrderYear    -- anno degli ordini
FROM Orders                                         -- emessi oggi
WHERE DATE(OrderDate) = current_date()
```

COALESCE prende più espressioni e restituisce la prima non `NULL`. Utile per dare un valore di default ai nulli:

```
SELECT Number, coalesce(Mobile, PhoneHome) FROM Employee
SELECT Name, Surname, coalesce(Dept, 'None') FROM Employee
```

NULLIF confronta il primo argomento con il secondo: restituisce `NULL` se coincidono, altrimenti il primo argomento.

```
SELECT Surname, nullif(Dept, 'Unknown') FROM Employee
```

CASE logica “if-then-else” il cui risultato dipende dal contenuto delle tabelle. Tassa dei veicoli immatricolati dopo il 1975 in base al tipo:

```
SELECT PlateNo,
       (CASE Type
        WHEN 'Car' THEN 2.58 * KWatt
        WHEN 'Moto' THEN (22.00 + 1.00 * KWatt)
        ELSE null
        END) AS Tax
FROM Vehicle
WHERE RegistrationYear > 1975
```

5.7 Sicurezza: privilegi

SQL permette di assegnare a ogni utente specifici **privilegi** (lettura, scrittura, ...) su tutto il DB o su parti di esso (relazioni, attributi, viste, domini). Esiste almeno un amministratore di default (es. `_system`) con tutti i privilegi; chi crea una risorsa ottiene automaticamente tutti i privilegi su di essa.

Un **privilegio** è descritto da la risorsa, l'utente che lo concede, l'utente che lo riceve, l'operazione, e se può essere *propagato* ad altri.

Privilegi SQL

insert

inserire nuove tuple;

update

modificare tuple esistenti;

delete

cancellare tuple;

select

leggere una risorsa;

references

definire vincoli di integrità referenziale verso la risorsa;

usage

usare una definizione (es. un dominio).

GRANT

```
GRANT < Privilegi | ALL PRIVILEGES > ON Risorsa TO Utenti [ WITH GRANT OPTION ]
```

```
GRANT SELECT ON Department TO Jack
```

WITH GRANT OPTION permette all'utente di propagare il privilegio.

REVOKE

```
REVOKE Privilegi ON Risorsa FROM Utenti [ RESTRICT | CASCADE ]
```

I privilegi devono essere revocati dallo stesso utente che li ha concessi.

- **RESTRICT** (default): la revoca non coinvolge gli utenti che hanno ricevuto il privilegio dall'utente corrente;
- **CASCADE**: la revoca si estende anche a loro (attenzione: "reazione a catena").

Discussione

- il sistema deve nascondere *senza dare indizi* le parti del DB non accessibili: se la tabella **Employees** non esiste o se esiste ma l'utente non vi ha accesso, il messaggio deve essere lo stesso;
- per mostrare a un utente solo alcune tuple si definisce una **vista** con un predicato di selezione e si concede il privilegio sulla vista.

5.8 RBAC

SQL-3 introduce il **Role-Based Access Control**: un *ruolo* è un contenitore di privilegi, concessi con **GRANT**. In ogni momento un utente ha i privilegi concessi a lui individualmente più quelli del ruolo attivo.

```
CREATE ROLE Employee;           -- crea il ruolo
GRANT CREATE TABLE TO Employee; -- privilegio al ruolo
GRANT Employee TO user1;        -- ruolo a un utente
REVOKE CREATE TABLE FROM Employee; -- revoca dal ruolo
SET ROLE Employee               -- attiva il ruolo per l'utente corrente
```

5.9 Transazioni

Una **transazione** è un programma in esecuzione che costituisce un'unità logica di elaborazione (operazione atomica). Proprietà **ACID**:

Atomicità

la transazione è eseguita tutta o per niente. Bonifico da A a B: o il denaro viene tolto da A e accreditato a B, o non si fa nulla.

Consistenza

se eseguita interamente e senza interferenze, porta il DB da uno stato consistente a un altro. Durante l'esecuzione i vincoli possono essere violati, ma se alla fine ci sono ancora violazioni la transazione va annullata (*abort*).

Isolamento

anche con molte transazioni concorrenti, l'esecuzione di una non deve subire interferenze dalle altre (es. due bonifici “contemporanei” sullo stesso conto).

Durabilità

le modifiche di una transazione andata a buon fine (*committed*) devono persistere e non essere perse, anche in caso di guasti hardware o software.

Transazioni in SQL una transazione inizia con **START TRANSACTION** (opzionale nella maggior parte dei DBMS) e termina con:

- **COMMIT [WORK]**: le operazioni vengono salvate nel DB;
- **ROLLBACK [WORK]**: le operazioni vengono annullate e il DB torna allo stato sicuro precedente.

Molti DBMS hanno l'**autocommit**: ogni istruzione è una transazione a sé.

```
START TRANSACTION;                                -- opzionale
UPDATE BankAccount SET Balance = Balance - 10 WHERE AccountNumber = 42177;
UPDATE BankAccount SET Balance = Balance + 10 WHERE AccountNumber = 12202;
COMMIT WORK;
```

5.10 Controllo di concorrenza: anomalie

Il DBMS esegue molte transazioni in parallelo (alternandone le operazioni) per sfruttare le risorse. Senza controllo, l'esecuzione concorrente produce **anomalie**. Notazione: $r_i(x)$ e $w_i(x)$ sono lettura e scrittura di x da parte di T_i , c_i e a_i sono commit e abort.

Perdita di aggiornamento (lost update)

$r_1(x) r_2(x) w_1(x) w_2(x)$: entrambe leggono x , poi la scrittura di T_2 cancella quella di T_1 (conflitto WW).

Lettura sporca (dirty read)

$w_1(x) r_2(x) a_1$: T_2 legge un valore scritto da T_1 che poi fa abort, quindi un valore mai esistito (conflitto WR).

Lecture inconsistenti (non ripetibili)

$r_1(x) w_2(x) c_2 r_1(x)$: T_1 legge due volte x e ottiene valori diversi (conflitto RW).

Aggiornamento fantasma

T_1 legge dati che, presi insieme, violano un vincolo perché T_2 ne ha aggiornati solo una parte (es. somma $x + y + z$ letta a metà di un trasferimento).

Inserimento fantasma

T_1 calcola un aggregato su un insieme di tuple (es. media degli stipendi), T_2 *inserisce* una tupla che soddisfa la condizione, e una seconda lettura di T_1 dà un risultato diverso.

5.11 Schedule e serializzabilità

Uno **schedule** è una sequenza di operazioni di più transazioni che rispetta l'ordine interno di ciascuna. È **seriale** se le transazioni sono eseguite una dopo l'altra senza alternanza: uno schedule seriale non ha anomalie. Lo scheduler deve accettare solo schedule **serializzabili**, cioè equivalenti a uno schedule seriale.

- Due operazioni sono in **conflitto** se appartengono a transazioni diverse, operano sullo stesso oggetto e almeno una è una scrittura (coppie RW, WR, WW).
- **Conflict-serializzabilità** (CSR): uno schedule è CSR se è conflict-equivalente (stesse coppie di conflitto, nello stesso ordine) a uno seriale.
- **Grafo dei conflitti**: un nodo per transazione e un arco $T_i \rightarrow T_j$ se un'operazione di T_i precede un'operazione di T_j in conflitto con essa. Lo schedule è CSR *se e solo se il grafo è aciclico*.

Esempio: in $w_1(A) w_2(A) w_2(B) w_1(B)$ ci sono $T_1 \rightarrow T_2$ (su A) e $T_2 \rightarrow T_1$ (su B): il grafo ha un ciclo, quindi lo schedule non è serializzabile.

5.12 Lock e locking a due fasi (2PL)

Nei DBMS reali la serializzabilità si ottiene con i **lock**: prima di leggere una risorsa si chiede un *lock condiviso* (S , read lock), prima di scriverla un *lock esclusivo* (X , write lock). Compatibilità: più lock S possono coesistere, mentre X è incompatibile con tutto. Se il lock non è concesso, la transazione resta in *attesa*.

2PL (locking a due fasi)

una transazione, dopo aver rilasciato un lock, non può acquisirne altri: c'è una fase *crescente* e poi una *calante*. Garantisce la serializzabilità.

2PL stretto

i lock si rilasciano solo dopo commit o abort. Evita anche le letture sporche e l'effetto domino degli abort a cascata. È quello usato in pratica.

Esempio di lettura sporca risolta con 2PL stretto:

$$X_1(x) w_1(x) S_2(x) [\text{attesa}] a_1 [\text{rilascio}] r_2(x) c_2$$

T_2 legge x solo dopo che T_1 ha terminato, quindi legge il valore corretto.

Deadlock con i lock due transazioni possono aspettarsi a vicenda (T_1 attende un lock di T_2 e viceversa). Si risolve con *timeout*, con la rilevazione dei cicli nel *grafo delle attese* o con la prevenzione; in ogni caso una transazione viene abortita e rieseguita.

5.13 Livelli di isolamento in SQL

SQL permette di rinunciare a parte dell'isolamento per guadagnare in prestazioni (`SET TRANSACTION ISOLATION LEVEL ...`). In tutti i livelli i *lock in scrittura* sono tenuti fino al commit (2PL stretto sulle scritture), quindi la perdita di aggiornamento “pura” su scritture è sempre evitata. Cambia la gestione delle letture:

Livello	Letture	Anomalie ancora possibili
READ UNCOMMITTED	nessun lock in lettura	lettura sporca, letture inconsistenti, fantasmi
READ COMMITTED	lock in lettura brevi	letture inconsistenti, aggiornamento e inserimento fantasma
REPEATABLE READ	lock in lettura lunghi (sulle tuple)	inserimento fantasma
SERIALIZABLE	lock lunghi + lock di predicato	nessuna

Per scegliere il *livello minimo* per uno schedule: lettura sporca $\Rightarrow$ `READ COMMITTED`; letture non ripetibili o perdita di aggiornamento con letture precedenti $(r_1 r_2 w_1 w_2) \Rightarrow$ `REPEATABLE READ`; inserimento fantasma $\Rightarrow$ `SERIALIZABLE`.

5.14 Affidabilità: log e ripresa dai guasti

Il **gestore dell'affidabilità** garantisce atomicità e durabilità usando il **log**, un file sequenziale su memoria stabile che registra le azioni delle transazioni:

- record di transazione: $B(T)$ begin, $C(T)$ commit, $A(T)$ abort; $U(T, O, BS, AS)$ update, con *before state* e *after state*; $I(T, O, AS)$ insert; $D(T, O, BS)$ delete;
- record di sistema: **checkpoint** $CK(T_1, \dots, T_n)$ con le transazioni attive in quel momento, e **dump**, una copia completa del DB.

Regole fondamentali:

WAL (write-ahead log)

la parte *before state* del log va scritta su disco *prima* del dato corrispondente: così si può sempre fare UNDO.

Commit-precedenza

la parte *after state* del log va scritta *prima* del commit: così si può sempre fare REDO.

Il commit avviene quando il record $C(T)$ è scritto in modo sincrono nel log: da lì la transazione è definitiva.

Ripresa a caldo (guasto di sistema, memoria centrale persa): si risale il log fino all'ultimo checkpoint, si costruiscono l'insieme UNDO (le transazioni attive senza commit) e l'insieme REDO (le transazioni con commit), si *disfano* le prime ripercorrendo il log all'indietro e si *rifanno* le seconde in avanti.

Ripresa a freddo (guasto di dispositivo, disco perso): si ripristina l'ultimo *dump*, si rifanno in avanti tutte le azioni del log fino al guasto, poi si esegue una ripresa a caldo.

5.15 ARIES

ARIES è l'algoritmo di recovery più diffuso. Ogni record di log ha un *LSN* (numero di sequenza) e ogni pagina conserva il *pageLSN*, l'ultimo aggiornamento che contiene. Al riavvio esegue tre fasi:

1. **Analisi**: dal checkpoint determina le transazioni attive al momento del crash (le *loser*) e le pagine forse non scritte su disco.
2. **REDO** (“ripetere la storia”): riesegue in avanti tutte le operazioni del log il cui effetto non è già sulla pagina ($LSN > pageLSN$), comprese quelle delle *loser*.
3. **UNDO**: annulla all'indietro le operazioni delle *loser*, scrivendo nel log un **CLR** (compensation log record) per ogni operazione annullata, così un nuovo crash non le annulla due volte.

Esempio. $A = 10$, un'unica transazione senza commit esegue 4 volte $A := A + 2$. Il log viene scritto su disco dopo l'operazione 1 e dopo la 3, A viene scritto su disco dopo la 1, poi c'è un crash dopo la 4.

- Log su disco: **1, 2, 3** (la 4 era solo in memoria). A su disco vale **12**.
- REDO: si rieseguo **2, 3** (la 1 è già su disco): $A = 16$.
- UNDO (la transazione non ha fatto commit): **3', 2', 1'**. Log finale: 1 2 3 3' 2' 1'; A torna a **10**.

6 Modellazione concettuale dei dati

6.1 Perché i modelli concettuali

Proviamo a costruire un DB relazionale direttamente dal modello logico:

- da dove si comincia?
- si rischia di perdersi subito nei dettagli;
- bisogna definire da subito i legami fra le tabelle;
- il modello relazionale è troppo “rigido” per modellare.

I modelli concettuali invece:

- permettono di ragionare sulla realtà di interesse con un “modello” *indipendente dall'implementazione*;
- permettono di definire le classi di oggetti di interesse e i loro legami;
- forniscono rappresentazioni grafiche efficaci (utili anche come documentazione).

6.2 Progettazione di basi di dati

La progettazione è una delle attività del processo di sviluppo di un sistema informativo, e va vista nel contesto più ampio del **ciclo di vita** dei sistemi informativi: l'insieme delle attività svolte in sequenza da analisti, progettisti e utenti nello sviluppo e nell'uso del sistema. È un processo iterativo (“cicli”).

Fasi del ciclo di vita

Studio di fattibilità

definizione di costi e priorità.

Raccolta e analisi dei requisiti

studio delle proprietà del sistema.

Progettazione

di dati e funzioni.

Realizzazione (implementazione)

Validazione e collaudo

verifica del modello, debugging.

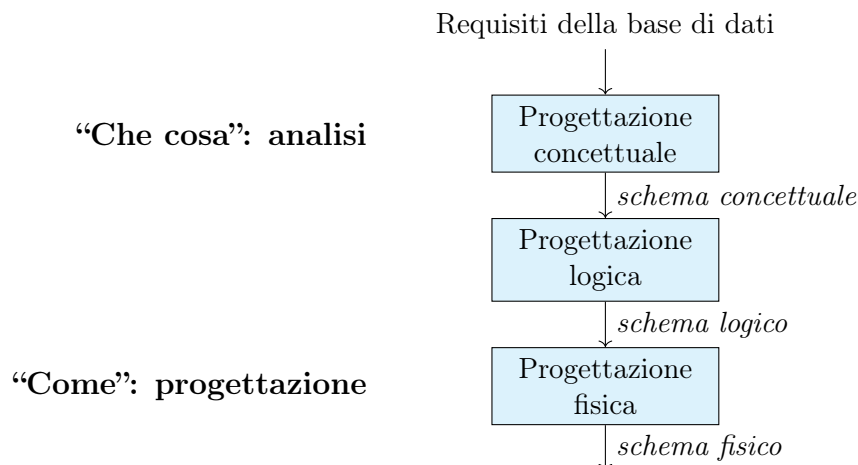
Funzionamento

il sistema è in esercizio.

Dati e applicazioni la progettazione di un SI riguarda due aspetti: progettazione dei **dati** e delle **applicazioni**. Ma i dati hanno un ruolo centrale: sono *più stabili*.

Metodologia di progettazione i progetti di qualità seguono sistematicamente una metodologia, fatta di: suddivisione in fasi separate, criteri di scelta, modelli di rappresentazione, soluzioni generali e facilità d'uso.

6.3 Fasi della progettazione



Il risultato di ogni fase è uno *schema* usato nella fase successiva: schema concettuale, logico, fisico.

Modello dei dati insieme di costrutti usati per organizzare i dati di interesse e descriverne le operazioni. La componente fondamentale sono i **costruttori** (come i costruttori di tipo nei linguaggi di programmazione): es. il modello relazionale ha il costruttore *relazione* per insiemi omogenei di tuple.

Schemi e istanze in ogni base di dati c'è uno **schema**, invariante nel tempo, che ne descrive la struttura (aspetto *intensionale*; nel relazionale: gli attributi delle relazioni) e un'**istanza**, i valori attuali, che possono cambiare anche molto rapidamente (aspetto *estensionale*; nel relazionale: l'insieme di tuple).

Due tipi di modelli

- **logici**: organizzano i dati nei DBMS; livello di astrazione usato dai programmi (es. framework di persistenza); indipendenti dalla struttura fisica. Es.: relazionale, a grafo, gerarchico, a oggetti;
- **concettuali**: rappresentano i dati indipendentemente dal sistema, cercando di descrivere i concetti del mondo reale; usati nelle fasi preliminari della progettazione. Il più diffuso è il modello **Entity-Relationship**.

6.4 Il modello Entity-Relationship

È il modello concettuale più usato (ne esistono molte versioni simili). Costrutti: **entità**, **relazione** (relationship), **attributo**, **identificatore**, **generalizzazione**, ...

6.5 Entità

Classe di oggetti (cose, persone, luoghi) della realtà di interesse con proprietà comuni ed *esistenza autonoma*. Es.: impiegato, città, conto corrente, ordine, fattura.

- *schema*: la classe omogenea di oggetti;
- *istanza* (occorrenza): un elemento della classe (l'oggetto, non i dati che lo descrivono).

Nello schema ER le entità non rappresentano le singole istanze ("astrazione"). Si rappresentano con un rettangolo; ogni entità ha un nome univoco nello schema, espressivo, per convenzione al *singolare*.



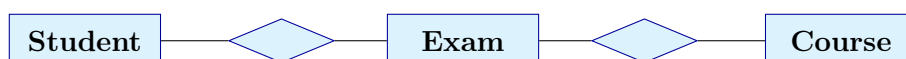
6.6 Relazioni (relationship)

Legame logico fra due o più entità del dominio (detto anche associazione, correlazione). Es.: *Residenza* fra Persona e Città, *Esame* fra Studente e Corso. Si rappresentano con un rombo; nome univoco, espressivo, se possibile un *sostantivo* singolare invece di un verbo.



Istanze un'istanza di relazione binaria è una *coppia* di istanze di entità, una per ciascuna entità coinvolta; un'istanza di relazione *n*-aria è una *n*-upla. **Non possono esserci istanze ripetute** (coppie o tuple uguali) in una relazione: è un insieme.

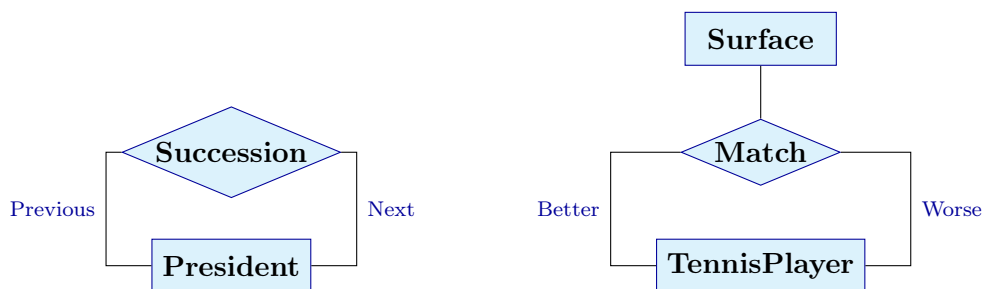
Attenzione con Student–Exam–Course uno studente non può sostenere due volte lo stesso esame per un corso (E1 ed E5 sarebbero la stessa coppia (S1, C1)). Se il requisito è “uno studente sostiene più volte l'esame dello stesso corso”, la soluzione è **promuovere la relazione a entità**: Exam diventa un'entità collegata a Student e a Course da due relazioni.



Più relazioni fra le stesse entità possibili: es. Workplace e Residence fra Employee e City.

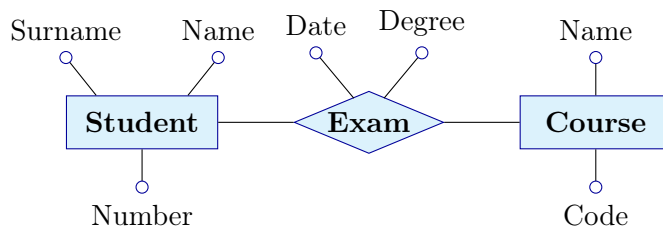
Relazioni n -arie es. Provision fra Provider, Supply e Dept.: ogni istanza è una tripla (fornitore, prodotto, dipartimento).

Relazioni ricorsive coinvolgono due volte la stessa entità (es. Acquaintance su Person). Se la relazione non è simmetrica servono i **ruoli**: Succession su President con ruoli *Previous* e *Next*. Possono essere anche ternarie: Match fra TennisPlayer (ruoli *Better*, *Worse*) e Surface.



6.7 Attributi

Proprietà di entità o relazioni: associa a ogni istanza un valore appartenente a un insieme detto **dominio**. Si disegnano come pallini collegati all'elemento.



Gli attributi di una relazione descrivono l'istanza della relazione: es. (S1, C1) ha data 25/07/2004 e voto 26.

Attributi composti raggruppano attributi della stessa entità o relazione con significato o uso affine: es. *Via*, *Numero*, *CAP* formano *Indirizzo* (si disegna un ovale).

6.8 Cardinalità delle relazioni

Coppia di valori associata a ogni entità che partecipa a una relazione: specifica il numero **minimo** e **massimo** di istanze della relazione a cui ogni istanza dell'entità può partecipare.



Un impiegato ha almeno un incarico e al più 5; un incarico può essere assegnato al più a 50 impiegati, ma anche a nessuno.

Per semplicità si usano tre simboli:

- 0 e 1 per la cardinalità minima: 0 = partecipazione *opzionale*, 1 = partecipazione *obbligatoria*;

- 1 e N per la massima: N non pone alcun limite.

Es. **Student** (1,1) **Residence** (0,N) **City**: ogni studente risiede in esattamente una città, una città ha zero o più studenti.

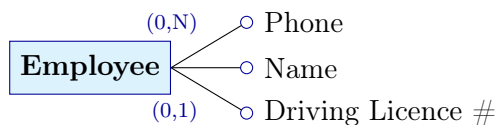
Tipi di relazioni rispetto alle cardinalità *massime*:

- **molti a molti**: Student (0,N) Exam (0,N) Course; Mountain (0,N) Climb (1,N) Climber; Pilot (1,N) Drives (1,N) Airplane;
- **uno a molti**: People (0,1) Job (0,N) Firm; Cinema (1,1) Place (0,N) City; Town (1,1) PartOf (1,N) District;
- **uno a uno**: Person (0,1) Holds (0,1) Pacemaker; PersonWithPacemaker (1,1) Holds (0,1) Pacemaker; PersonWithPacemaker (1,1) Holds (1,1) ImplantedPacemaker.

Attenzione al *verso* delle uno-a-molti; le relazioni con partecipazione obbligatoria da entrambi i lati sono molto rare.

6.9 Cardinalità degli attributi

Si possono associare cardinalità anche agli attributi, per indicare *opzionalità* (informazione parziale) o attributi *multivalore*. Se omessa vale (1,1).

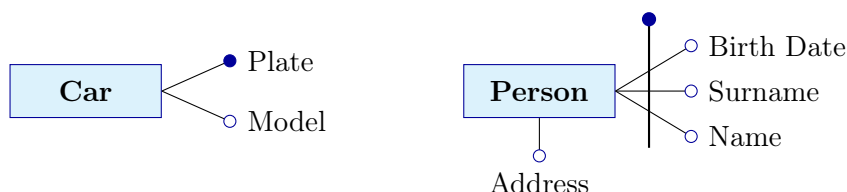


6.10 Identificatori

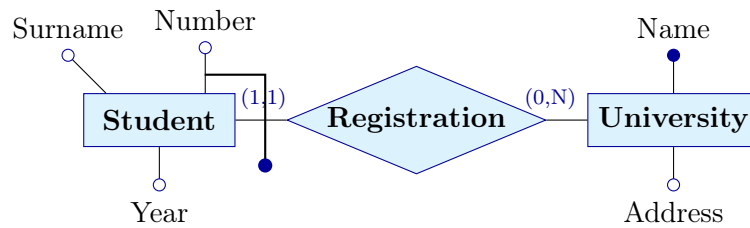
Strumento per identificare univocamente le istanze di un'entità. Sono formati da:

- attributi dell'entità: identificatore **interno**;
- (attributi +) entità esterne raggiungibili tramite relazioni: identificatore **esterno**.

Identificatori interni *Plate* identifica *Car* (non esistono due auto con la stessa targa). Se l'identificatore è composto da più attributi si disegna una linea che li unisce terminata da un pallino pieno: *Name*, *Surname*, *Birth Date* identificano *Person* (assumendo che non esistano omonimi nati lo stesso giorno).



Identificatori esterni *Number* e l'entità *University* formano l'identificatore di *Student*: possono esistere studenti con la stessa matricola in università diverse.



Osservazioni

- ogni entità deve avere *almeno un* identificatore, ma in generale può averne più d'uno;
- un identificatore esterno è possibile solo attraverso una relazione a cui l'entità da identificare partecipa con cardinalità **(1,1)**;
- le relazioni non hanno identificatori: è meglio associare attributi alle entità che alle relazioni.

Attenzione piccole modifiche a cardinalità e identificatori possono cambiare il significato dell'intero schema. Nello schema impiegati-dipartimenti-sedi (sotto), se *Dept.* è identificato da *Name* + *Office* con Composition (1,1) lato *Dept.* e (1,N) lato *Office*, ogni dipartimento ha una sola sede e possono esistere dipartimenti omonimi in sedi diverse. Se invece *Office* è identificata da *City* + *Dept.* (Composition (1,N) lato *Dept.*, (1,1) lato *Office*), ogni sede appartiene a un solo dipartimento e nella stessa città possono esserci molte sedi di dipartimenti diversi.

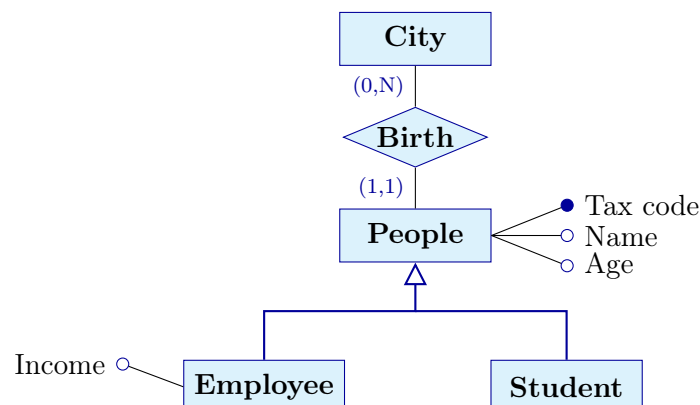
6.11 Generalizzazioni

Legano una o più entità $E_1, \dots, E_n$ a un'entità E che le comprende come casi particolari: E è una **generalizzazione** di $E_1, \dots, E_n$; le E_i sono **specializzazioni** di E . Si rappresentano con una freccia verso il genitore.

Proprietà se E (genitore) generalizza $E_1, \dots, E_n$ (figlie):

- ogni proprietà di E è significativa per le E_i ;
- ogni istanza di un' E_i è anche istanza di E .

Ereditarietà tutte le proprietà dell'entità genitore (attributi, relazioni, generalizzazioni) sono *ereditate* dalle figlie e non vanno rappresentate esplicitamente.

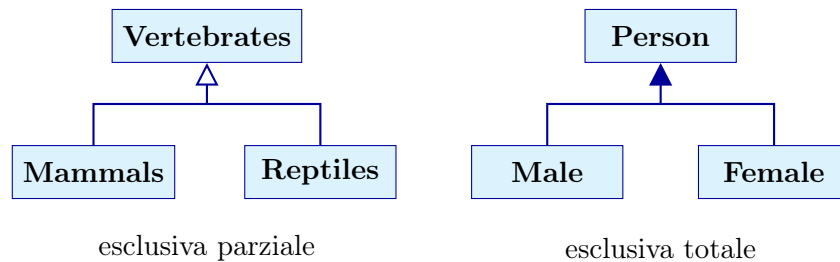


Employee e *Student* ereditano Tax code, Name, Age e la relazione Birth.

Tipi di generalizzazione

- **totale** se ogni istanza del genitore è istanza di almeno una figlia, **parziale** altrimenti;
- **esclusiva** (disjoint) se ogni istanza del genitore è istanza di al più una figlia, **sovrapposta** (overlapping) altrimenti.

Consideriamo (senza perdita di generalità) solo generalizzazioni esclusive, distinguendo totali (freccia piena) e parziali (freccia vuota).



Altre proprietà

- si possono avere gerarchie a più livelli e più generalizzazioni allo stesso livello;
- un'entità può far parte di più gerarchie, come genitore e/o figlia;
- una generalizzazione con una sola figlia si chiama **subset**;
- alcune configurazioni non hanno senso;
- il genitore di una generalizzazione può non avere identificatore, purché ... (lo erediti/ottenga altrimenti).

Esercizio le persone hanno codice fiscale, cognome ed età; per gli uomini interessa sapere se hanno svolto il servizio militare; gli impiegati hanno uno stipendio e possono essere dirigenti, funzionari o progettisti (alcuni progettisti sono responsabili di progetto); gli studenti (che non possono essere impiegati) hanno una matricola; alcune persone non sono né impiegati né studenti.

Soluzione: *People* (Tax code, Surname, Age) con due generalizzazioni: *Man* (Military) / *Woman* **totale**; *Employee* (Income) / *Student* (Matr.) **parziale**. *Employee* si specializza (totale) in *SwArchitect*, *Manager*, *Clerk*; *SwArchitect* ha il subset *In Charge*.

6.12 Documentazione degli schemi concettuali

Lo schema ER va corredato da:

- un **dizionario dei dati** (entità e relazioni);
- i **vincoli non esprimibili** nello schema.



Entità	Descrizione	Attributi	Identificatore
Employee	impiegato di un dipartimento	Code, Name, Surname	Code
Project	progetti di un dipartimento	Name, Budget	Name
Department	struttura dell'azienda	Name, Phone	Name, Office
Office	sede di un dipartimento	City, Address	City

Relazione	Descrizione	Componenti	Attributi
Management	direzione di un dipartimento	Employee, Dept.	Date
Affiliation	afferenza a un dipartimento	Employee, Dept.	
Attendance	partecipazione a un progetto	Employee, Project	
Composition	composizione dei dipartimenti	Dept., Office	

1. il direttore di un dipartimento deve afferire a quel dipartimento;
2. un impiegato non deve avere uno stipendio maggiore del direttore del suo dipartimento;
3. un dipartimento con sede a Roma deve essere diretto da un impiegato con almeno 10 anni di anzianità;
4. un impiegato che non afferisce a nessun dipartimento non deve partecipare ad alcun progetto.

UML2 può sostituire ER per la modellazione dei dati, usando i *class diagram*: cambia la rappresentazione grafica, l'approccio resta lo stesso.

ER	UML
entità	classe (rettangolo con nome e attributi)
relazione binaria	associazione (linea con nome)
attributi di relazione	<i>association class</i> (classe collegata alla linea)
relazione n -aria	rombo + <i>association class</i> , oppure meglio <i>reificazione</i>
cardinalità (min, max)	molteplicità $\text{min}.. \text{max}$ (scritta sul lato <i>opposto</i>)
identificatore interno	attributi marcati {id}
identificatore esterno	stereotipo «Identifying» sull'associazione
generalizzazione	freccia a triangolo vuoto con {total/partial, disjoint}

Associazioni con molteplicità

- **Order 1 -- Sell -- 0..1 Bill**: un ordine ha da 0 a 1 fattura; una fattura ha esattamente 1 ordine;
- **Person * -- Residence -- City**: nessuna molteplicità vale 1..1, quindi una persona risiede in esattamente una città; una città ha da 0 a N residenti;
- **Tourist * -- Reservation -- 1..* Hotel**: un turista prenota da 1 a N hotel; un hotel è prenotato da 0 a N turisti.

Reificazione le associazioni n -arie sono poco comuni: conviene reificarle in una classe (es. **Provision** con attributo **Quantity**) collegata alle classi originali con associazioni binarie.

Aggregazione e composizione

- **aggregazione** (rombo vuoto): un *Technician* appartiene a un *Team*, ma può essere rappresentato indipendentemente dal team;
- **composizione** (rombo pieno): un'*Agency* appartiene a una *Firm* e non può esistere indipendentemente da essa.

Esempio lo schema impiegati–dipartimenti in UML: *Employee* (Code {id}, Surname, Income, Age), *Dept.* (Name {id}, Phone 1..*), *Project* (Name {id}, Budget, Delivery Date 0..1), *Office* (City {id}, Address); *Affiliation* e *Attendance* come association class; *Composition* come composizione con stereotipo «Identifying»; i vincoli non esprimibili come note.

7 Progettazione concettuale

7.1 Requisiti e progettazione concettuale

La progettazione concettuale (il “che cosa”) comprende attività interconnesse:

- acquisizione (raccolta) dei requisiti;
- analisi dei requisiti;
- costruzione dello schema concettuale;
- costruzione del glossario.

7.2 Requisiti

Fonti possibili

- **utenti e committenti**, tramite interviste o documentazione apposita (*ad hoc*);
- **documentazione esistente**: normative (leggi, regolamenti di settore), regolamenti interni, procedure aziendali, realizzazioni preesistenti;
- **modulistica**.

Acquisizione e analisi l'acquisizione dei requisiti è un'attività difficile e non standardizzabile. Si parte raccogliendo i primi requisiti, che spesso vanno poi raffinati con ulteriori acquisizioni.

Interviste

- utenti diversi forniscono informazioni diverse;
- gli utenti di livello più alto nell'organizzazione hanno spesso una visione più ampia ma meno dettagliata;
- le interviste portano spesso a raffinamenti successivi (es. altre interviste).

Suggerimenti: effettuare spesso verifiche di comprensione e coerenza; usare esempi (casi generali e casi limite); chiedere definizioni e classificazioni; chiedere quali aspetti sono essenziali e quali marginali.

Documentazione descrittiva regole pratiche:

- scegliere il giusto livello di astrazione;
- standardizzare la struttura delle frasi;
- suddividere le frasi articolate;
- separare le frasi sui *dati* da quelle sulle *funzioni*.

Termini e concetti regole generali:

- costruire un **glossario** dei termini;
- individuare omonimi e sinonimi e unificarli in un unico termine;
- rendere espliciti i riferimenti fra termini;
- riorganizzare le frasi per concetti.

7.3 Esempi di requisiti

Biblioteca si vogliono automatizzare i dati sui riferimenti bibliografici. Per ogni riferimento si vuole un codice identificativo di 7 caratteri (iniziali degli autori, anno di pubblicazione, carattere di disambiguazione). Un riferimento può essere una *monografia* (interessano editore, data e luogo di pubblicazione) o un *articolo su rivista* (nome della rivista, volume, numero, pagine, anno); per entrambi interessano i nomi degli autori.

Società di formazione (esempio guida)

- si vuole una base di dati di una società che eroga corsi, di cui si vogliono rappresentare i dati dei partecipanti e dei docenti;
- per i partecipanti (~5000) si rappresentano codice, codice fiscale, cognome, età, sesso, luogo di nascita, nome dei datori di lavoro attuali, posti dove hanno lavorato in precedenza con periodo (data di inizio e fine), indirizzo e telefono, corsi frequentati (corsi attuali e passati, ~200) con il giudizio finale;
- si rappresentano i seminari frequentati e, per ogni giorno, dove e quando si tengono le lezioni;
- i corsi hanno un codice, un titolo e possono avere varie edizioni con data di inizio e fine e numero di partecipanti;
- se un partecipante è *libero professionista* si rappresentano l'area di interesse e, se lo possiede, il titolo professionale; se è *dipendente* si rappresentano livello e posizione ricoperta;
- dei datori di lavoro (attuali e passati) si rappresentano nome, indirizzo e telefono;
- per i docenti (~300) si rappresentano cognome, età, luogo di nascita, nome del corso che insegnano, corsi insegnati in passato e corsi che possono insegnare, tutti i recapiti telefonici. I docenti possono essere dipendenti interni o collaboratori esterni.

Glossario

Termine	Descrizione	Sinonimi	Collegamenti
Partecipante	persona che partecipa ai corsi	Studente	Corso, Società
Docente	docente dei corsi; può essere esterno	Insegnante	Corso
Corso	corso organizzato; può avere più edizioni	Seminario	Docente
Società	ente presso cui i partecipanti lavorano o hanno lavorato	Posti	Partecipante

Strutturazione dei requisiti si riorganizzano le frasi in gruppi omogenei (frasi di carattere generale, sui partecipanti, su specifici tipi di partecipanti, sui datori di lavoro, sui corsi, sui docenti), sostituendo i sinonimi con i termini del glossario (studente → partecipante, insegnante → docente, seminario → corso, posti → società).

7.4 Dai requisiti ai costrutti ER

Quale costrutto usare per un termine dei requisiti?

Entità

se il termine ha proprietà significative e descrive oggetti con esistenza autonoma.

Attributo

se il termine è semplice e non ha proprietà.

Relazione

se il termine correla due o più concetti.

Generalizzazione

se un termine è un caso più generale di un altro.

7.5 Pattern di progettazione

Soluzioni progettuali consolidate a problemi comuni (come i design pattern dell'ingegneria del software).

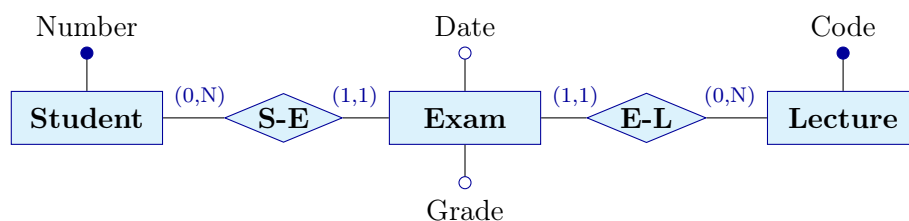
Reificazione di attributo di entità un attributo come *Company* di *Employee* va spesso trasformato in un'entità *Company* collegata da una relazione *Job* (*Employee* (1,1), *Company* (1,N)): la società è in genere un concetto a sé, specialmente se coinvolta in altre relazioni.

Part-of le relazioni uno a molti possono rappresentare relazioni *parte di*:

- **composizione:** un cinema è fatto di sale, ogni sala appartiene a un cinema (*Cinema* (1,N) *MadeOf* (1,1) *Hall*, con *Hall* identificata esternamente da *Number* + *Cinema*);
- **aggregazione:** una squadra è fatta di esperti (*Team* (1,N) *MadeOf* (0,1) *Expert*): l'esperto esiste anche senza squadra.

Instance-of a volte servono due entità distinte: una per il concetto *astratto* e una per la sua realizzazione concreta. Es. *Flight* (*Code*, *Source*, *Destination*, *Time*) (1,N) *InstOf* (1,1) *Scheduled* (*Date*), con *Scheduled* identificata da *Date* + *Flight*; analogamente *Tournament* – *Edition*.

Reificazione di relazione binaria se uno studente può sostenere più volte l'esame di un corso, la relazione *Exam* (*Date*, *Grade*) fra *Student* e *Lecture* diventa un'entità *Exam* collegata con S-E e E-L (lato *Exam* (1,1)); il suo identificatore può essere esterno (*Student* + *Lecture* + *Date*) oppure un codice interno.



Reificazione di relazione ricorsiva una partita potrebbe essere una relazione binaria ricorsiva fra squadre, ma due squadre possono giocare più partite: si reifica *Match* (*Date*, *Score*) collegata a *Team* con *Home* e *Visiting* (lato *Match* (1,1)).

Reificazione di attributo di relazione un musicista suona in un'orchestra uno strumento, ma lo stesso strumento può essere suonato molte volte: l'attributo *Instrument* di *Plays* diventa un'entità, e *Plays* viene reificata in un'entità collegata a *Player*, *Orchestra* e *Instrument*.

Caso particolare con le generalizzazioni si definisce un caso specifico di un'entità: *Manager* specializza *Employee* ed è l'unico a partecipare a *manage* con *Project* (non tutti gli impiegati dirigono un progetto).

Storicizzazione per mantenere la storia di un concetto si usa una generalizzazione in *Historic* (con data di scadenza) e *Current*, oppure due relazioni *CurrentEmployment* (0,1)/(1,N) e *PastEmployment* (0,N)/(1,N, con Begin/End) fra *Employee* e *Company*; o ancora un'entità *Employment* specializzata in *Current* e *Past*.

Evoluzione di un concetto le generalizzazioni servono anche a estendere un concetto quando il progetto è già in esercizio: un *Accepted project* (Founding, Start Date) richiede più informazioni degli altri progetti (in attesa o rifiutati).

Reificazione di relazione ternaria *Work* fra *Empl.*, *Office* e *Task*: gli impiegati lavorano su più compiti in più sedi. Si reifica *Work* (Span) in un'entità identificata dalle tre relazioni E-W, O-W, T-W (tutte (1,1) lato Work). Se invece un compito è svolto da un solo impiegato e in una sola sede, la reificazione si semplifica: *Empl.* (1,N) O-S (1,1) *Task* (1,1) S-I (1,N) *Office*.

7.6 Strategie di progettazione

Come tradurre i requisiti in schemi ER?

Top-down si parte da uno schema iniziale molto astratto e lo si raffina progressivamente (schema bozza $\rightarrow$ intermedio $\rightarrow$ finale) con **primitive di raffinamento**: un'entità diventa una relazione fra entità (*Exam* $\rightarrow$ Student–Exam–Lecture), si aggiungono attributi (*Employee* $\rightarrow$ Employee con Surname, Age, Wage), un'entità si specializza (*People* $\rightarrow$ Man/Woman). Vantaggio: si parte da una visione d'insieme; ma serve una visione globale fin dall'inizio, difficile per sistemi complessi.

Bottom-up si suddividono i requisiti in parti sempre più piccole, si costruisce uno schema per ciascuna e poi si **integrano** gli schemi nello schema finale. Primitive: da un requisito si ricava un'entità; si introduce una relazione fra entità esistenti; si introduce una generalizzazione fra entità esistenti (*People, Man, Woman* $\rightarrow$ gerarchia). Rischio: l'integrazione può essere difficile.

Inside-out si parte da un concetto importante e ci si espande “a macchia d'olio” verso i concetti vicini: *Employee* $\rightarrow$ + attributi $\rightarrow$ + *Dept.* con Supervision e Belonging $\rightarrow$ + *Project* con Enrollment $\rightarrow$ + *Office* con MadeOf.

Regola pratica: strategia mista

- si crea prima uno **schema scheletro** con le entità più rilevanti (le più citate o quelle indicate esplicitamente come importanti);
- poi si decompone lo schema;
- poi si raffina (bottom-up), si espande (top-down), si integra (inside-out).

7.7 Metodologia

1. **Analisi dei requisiti**: analizzare i requisiti ed eliminare le ambiguità; costruire il glossario; raggruppare i requisiti in frasi omogenee.
2. **Passo base**: definire uno schema scheletro con i concetti più rilevanti.

3. **Passo iterativo** (ripetere finché non si è soddisfatti): raffinare i concetti presenti sulla base dei requisiti; aggiungere concetti per descrivere requisiti non ancora rappresentati.
4. **Analisi di qualità** (ripetuta durante tutto il processo): verificare la qualità dello schema e modificarlo.

Qualità di uno schema ER correttezza, completezza, leggibilità, minimalità.

Con integrazione di schemi per progetti grandi: analisi dei requisiti, passo base, **decomposizione** (dei requisiti complessi secondo lo schema scheletro, o per aree di interesse, anche con acquisizioni separate), passo base e iterativo *per ogni sotto-schema/area*, **integrazione** dei sotto-schemi in uno schema complessivo usando lo scheletro come riferimento, analisi di qualità.

7.8 Esempio completo: società di formazione

Schema scheletro dalla frase generale (“corsi, partecipanti e docenti”): *Participant* –Presence– *Lecture* –Teaching– *Lecturer*.

Raffinamento: partecipanti e datori di lavoro *Participant* (Tax, Code, ...) è generalizzazione di *Employee* (Level, Position) e *Freelance* (Titolo prof., Area); *Employer* (Name, ...) è collegato a *Participant* da *PastEmpl* (0,N)–(0,N) e a *Employee* da *CurrEmpl* (1,1)–(0,N).

Raffinamento: corsi *Lecture* (Code, Title) (0,N) Kind (1,1) *Edition* (Start, End, #Part.), con *Edition* identificata da Start + *Lecture*; *Edition* (1,N) MadeOf (1,1) *Lesson* (Time, Room, Day), con *Lesson* identificata da Time, Room, Day.

Raffinamento: docenti *Lecturer* (Tax, Surname, Age, Place of Birth, Phone (1,N)) con generalizzazione in *Independent* e *Home*.

Integrazione nello schema scheletro: *Presence* diventa *PastPresence* (0,N)–(0,N) e *CurrentPresence* (0,1)–(0,N) fra *Participant* ed *Edition*; *Teaching* diventa *Past* (0,1)–(0,N) e *Current* (0,1)–(0,1) fra *Edition* e *Lecturer*, più *Duty* (1,N)–(1,N) fra *Lecture* e *Lecturer* (corsi che il docente può insegnare). Lo schema finale contiene *Participant* (con *Freelance*, *Employee*), *Employer*, *Lecture*, *Edition*, *Lesson*, *Lecturer* (con *Independent*, *Home*) e le relazioni viste.

8 Progettazione logica

8.1 Obiettivo

“Tradurre” lo schema concettuale in uno schema logico che rappresenti gli stessi dati in modo **corretto** ed **efficiente**.

- **ingresso:** schema concettuale, informazioni sul carico applicativo, modello logico;
- **uscita:** schema logico (relazionale, a oggetti, a grafo, ...) e documentazione associata.

Non è una semplice traduzione: alcuni aspetti non sono rappresentabili direttamente, e bisogna considerare anche le *prestazioni*.

Due fasi

1. **ristrutturazione dello schema ER** (input: schema ER e carico applicativo; output: schema ER ristrutturato);
2. **traduzione nel modello logico** (input: schema ristrutturato e modello logico; output: schema logico).

8.2 Ristrutturazione: prestazioni

Si ristruttura per semplificare la traduzione e ottimizzare le prestazioni. Nota: uno schema ER ristrutturato *non* è più uno schema concettuale in senso stretto.

Le prestazioni non si possono valutare con precisione su uno schema concettuale; si usano **indicatori** approssimati:

- **spazio**: numero di istanze previste;
- **tempo**: numero di istanze (di entità e relazioni) visitate durante un'operazione.

Tavola dei volumi per lo schema impiegati–dipartimenti–progetti:

Concetto	Tipo	Volume
Office	E	10
Dept.	E	80
Employee	E	2000
Project	E	500
MadeOf	R	80
Belonging	R	1900
Supervision	R	80
Enrollment	R	6000

Tavola degli accessi operazione: “trova tutti i dati di un impiegato, del dipartimento a cui appartiene e dei progetti a cui partecipa”. Si costruisce seguendo lo *schema di navigazione*:

Concetto	Costrutto	Accessi	Tipo	Ordine
Employee	Entità	1	L	1
Belonging	Relazione	1	L	2
Dept.	Entità	1	L	3
Enrollment	Relazione	3	L	4
Project	Entità	3	L	5

(3 = numero medio di progetti per impiegato: 6000/2000.)

Attività di ristrutturazione

1. analisi delle ridondanze;
2. eliminazione delle generalizzazioni;
3. partizionamento/accorpamento di entità e relazioni;
4. scelta degli identificatori primari.

8.3 Analisi delle ridondanze

Una **ridondanza** in uno schema ER è un'informazione significativa ma derivabile da altre. In questa fase si decide se mantenerle, eliminarle o crearne di nuove.

- **pro:** semplificano le interrogazioni;
- **contro:** appesantiscono gli aggiornamenti e occupano più spazio.

Forme di ridondanza

- attributi derivabili da altri attributi della stessa entità (o relazione): es. in *Invoice*, $Gross\ value = Net\ value + VAT$;
- attributi derivabili da attributi di altre entità (o relazioni): es. *Total* di *Purchase* derivabile da *Quantity* di *Composition* e *Price* di *Product*;
- attributi derivabili da operazioni di conteggio;
- relazioni derivabili dalla composizione di altre relazioni (in generale: **cicli** di relazioni). Es. *Teaching* (Student–Lecturer) è derivabile da *Attendance* (Student–Lecture) e *Tuition* (Lecture–Lecturer).

Esempio *Person* –*Stay*– *City*, con attributo ridondante *Inhabitants* di *City*. Volumi: *City* 200, *Person* 1 000 000, *Stay* 1 000 000. Operazioni:

- **op. 1:** memorizza una nuova persona con la sua città di residenza (500 volte al giorno);
- **op. 2:** stampa tutti i dati di una città, incluso il numero di abitanti (2 volte al giorno).

Con ridondanza

op. 1			
Person	E	1	S
Stay	R	1	S
City	E	1	L
City	E	1	S
op. 2			
City	E	1	L

Senza ridondanza

op. 1			
Person	E	1	S
Stay	R	1	S
op. 2			
City	E	1	L
Stay	R	5000	L

(5000 = $10^6/200$, residenti medi per città.) Contando le scritture **doppie**:

- **con** ridondanza: op. 1 = 1500 scritture + 500 letture al giorno; op. 2 trascurabile $\Rightarrow 3000 + 500 = \mathbf{3500}$ accessi al giorno;
- **senza** ridondanza: op. 1 = 1000 scritture; op. 2 = circa 10 000 letture $\Rightarrow 2000 + 10\,000 = \mathbf{12\,000}$ accessi al giorno.

Conviene mantenere la ridondanza (lo spazio in più è solo un intero per città).

8.4 Eliminazione delle generalizzazioni

Il modello relazionale non rappresenta direttamente le generalizzazioni, mentre entità e relazioni sì: si eliminano le gerarchie sostituendole con entità e relazioni. Data la generalizzazione E_0 (A01, A02; relazione R1 con E_3) con figlie E_1 (A11) ed E_2 (A21; relazione R2 con E_4), tre possibilità:

1. **accorpamento delle figlie nel genitore:** resta E_0 con attributi A01, A02, A11 (0,1), A21 (0,1) e un attributo *Kind* che indica il tipo; R2 si collega a E_0 con cardinalità minima 0;
2. **accorpamento del genitore nelle figlie:** restano E_1 (A01, A11, A02) ed E_2 (A01, A21, A02); R1 si sdoppia in R11 (con E_1) e R12 (con E_2). Possibile solo se la generalizzazione è *totale*;
3. **sostituzione della generalizzazione con relazioni:** restano tutte le entità, legate da relazioni RG1, RG2 (E_0 (0,1), E_i (1,1)); le figlie sono identificate esternamente da E_0 .

Quale scegliere si decide in base ai volumi e alle tavole degli accessi (contare solo gli accessi non basta). Regole generali:

1. conviene quando le operazioni accedono a genitore e figlie *insieme*;
2. conviene quando le operazioni accedono alle figlie *distintamente* fra loro;
3. conviene quando si accede alle figlie separatamente dal genitore.

Sono possibili anche soluzioni **ibride**, specie con gerarchie a più livelli: es. E_1 accorpata in E_0 (A11 (0,1), *Kind*) ed E_2 legata con RG2.

8.5 Partizionamento e accorpamento

Si ristruttura per ridurre il numero di accessi:

- attributi usati *separatamente* vengono separati;
- attributi usati *insieme* vengono raggruppati, anche se appartengono a entità/relazioni diverse.

Casi principali

- **partizionamento verticale di entità:** *Employee* (Code, Surname, Address, Day of birth, Level, Wage, Withholding tax) si divide in *Personal Data* (Code, Surname, Address, Day of birth) e *Corporation Data* (Level, Wage, Withholding tax) legate da una relazione (1,1)–(1,1), con *Corporation Data* identificata esternamente;
- **eliminazione di attributi multivalore:** *Office* (Name, Address, City, Phone (1,N)) → nuova entità *Phone* (Number) legata a *Office* da *Appliance* (Office (1,N), Phone (1,1));
- **accorpamento di entità:** *Person* (0,1) *Lives* (1,1) *Apartment* → un'unica entità *Person* con attributi No. (0,1) e Addr (0,1);
- **partizionamento orizzontale di relazioni:** *PartOf* fra *Player* e *Team* (con Bought, Cession (0,1)) si divide in *Current* (con Bought) e *Past* (con Bought, Cession).

8.6 Scelta degli identificatori primari

Operazione indispensabile per la traduzione nel modello relazionale. Criteri:

- assenza di opzionalità (nessun NULL);
- semplicità;
- utilizzo nelle operazioni più frequenti o importanti.

Se nessun identificatore soddisfa i requisiti si introducono nuovi attributi con **codici** generati appositamente.

8.7 Traduzione nel modello relazionale

Idea di base:

- le **entità** diventano relazioni (tabelle) sui loro attributi;
- le **relazioni** (relationship) diventano relazioni sui loro attributi più gli identificatori delle entità coinvolte.

Relazioni molti a molti *Employee* (Number, Surname, Wage) (0,N) *Enrollment* (StartDate) (1,N) *Project* (Code, Name, Budget):

```
Employee(Number, Surname, Wage)
Project(Code, Name, Budget)
Enrollment(Number, Code, StartDate)
```

con vincoli di integrità referenziale fra Number di *Enrollment* e la chiave di *Employee*, e fra Code di *Enrollment* e la chiave di *Project*. Conviene usare nomi più espressivi per le chiavi esterne: *Enrollment*(Employee, Project, StartDate).

Nota: questa traduzione non tiene conto della cardinalità *minima* delle molti a molti (si potrebbe solo con CHECK complessi e poco supportati).

Relazioni ricorsive *Product* (Code, Name, Cost) con *MadeOf* (Number) e ruoli MadeUp/Element:

```
Product(Code, Name, Cost)
MadeOf(MadeUp, Element, Number)
```

Relazioni n-arie *Supply* (Number) fra *Supplier* (VAT, Name), *Product* (Code, Type), *Department* (Name, Phone):

```
Supplier(VAT, Name)   Product(Code, Type)   Department(Name, Phone)
Supply(Supplier, Product, Department, Number)
```

Relazioni uno a molti *Player* (Surname, DayOfBirth, Role) (1,1) *Agreement* (Hire) (0,N) *Team* (Name, City, TeamColors). Traduzione “standard” (corretta):

```
Player(Surname, DayOfBirth, Role)
Agreement(SurnameP, BDayP, Team, Hire)
Team(Name, City, TeamColors)
```

(la chiave di *Agreement* è solo il giocatore: ognuno ha un solo contratto). Soluzione **meno ridondante**, accorpando la relazione nel lato (1,1):

```
Player(Surname, DayOfBirth, Team, Role, Hire)
      Team(Name, City, TeamColors)
```

con vincolo referenziale fra **Team** in **Player** e la chiave di **Team**. Se la cardinalità minima fosse 0, **Team** in **Player** dovrebbe ammettere NULL: questa traduzione rappresenta anche il caso (0,1), con $0 \rightarrow \text{NULL}$ ammessi, $1 \rightarrow \text{NULL}$ non ammessi.

Entità con identificatore esterno *Student* (Number, Surname, Course Year) (1,1) *Enrollment* (1,N) *University* (Name, City, Address), con *Student* identificato da *Number* + *University*:

```
Student(Number, University, Surname, CourseYear)
      University(Name, City, Addr)
```

La relazione è rappresentata dall'identificatore: ogni studente è iscritto a una sola università.

Relazioni uno a uno *Head* (Code, Surname, Wage) (1,1) *Supervision* (StartDate) (1,1) *Department* (Name, Office, Phone). Opzioni: accorpare la relazione a una delle due entità (da una parte o dall'altra), oppure accorpare tutto. Soluzione preferita:

```

    Head(Code, Surname, Wage)
Department(Name, Office, Phone, Head, StartDate)

```

un vincolo referenziale, nessun NULL. Altra soluzione: `Supervision(Head, Department, StartDate)` con due vincoli referenziali.

8.8 Esempio completo

Lo schema impiegati–dipartimenti–sedi (Employee, Dept., Project, Office con Supervision, Belonging, Enrollment, MadeOf; Dept. identificato da Name + Office) si traduce in:

```
Employee(Code, Surname, Department, Office, Date*)
    Dept. (Name, City, Phone, Head)
        Office(City, Road, Zip)
            Project(Name, Budget)
                Enrollment(Employee, Project)
```

(* = ammette NULL; *Belonging* è accorpata in **Employee**, *Supervision* in **Dept.**, l'attributo multivalore **Phone** si assume ristrutturato.)

Attenzione differenze apparentemente piccole di cardinalità e identificatori portano a significati molto diversi. Se *Office* è identificata da City + Dept. (MadeOf: Dept. (1,N), Office (1,1)), si ottiene invece:

```
Dept. (Name, City, Phone, Head)
Office (City, Dept., Road, Zip)
```

Strumenti esistono strumenti CASE commerciali che supportano tutte le fasi della progettazione (es. ERwin), fino alla generazione automatica delle istruzioni CREATE TABLE.

9 Normalizzazione

9.1 Forme normali

Una **forma normale** è una proprietà di una base di dati relazionale che ne garantisce la qualità, cioè l'assenza di certi difetti. Quando una relazione non è normalizzata:

- presenta *ridondanze*;
- si presta a comportamenti poco desiderabili durante gli aggiornamenti.

Le forme normali sono di solito definite sul modello relazionale, ma hanno senso anche in altri contesti (es. il modello ER).

La **normalizzazione** è la procedura che trasforma schemi non normalizzati in schemi che soddisfano una forma normale. Va usata come **tecnica di verifica** dei risultati della progettazione: *non* è una metodologia di progettazione.

9.2 Una relazione con anomalie

Employee	Wage	Project	Budget	Role
Jones	20	Mars	2	Technician
Smith	35	Jupiter	15	Designer
Smith	35	Venus	15	Designer
Williams	55	Venus	15	Chief
Williams	55	Jupiter	15	Consultant
Williams	55	Mars	2	Consultant
Brown	48	Mars	2	Chief
Brown	48	Venus	15	Designer
White	48	Venus	15	Designer
White	48	Jupiter	15	Director

Anomalie

Ridondanza

lo stipendio di ogni impiegato è ripetuto in tutte le sue tuple (idem il budget di ogni progetto).

Anomalia di aggiornamento

se lo stipendio di un impiegato cambia, bisogna modificarne tutte le occorrenze.

Anomalia di cancellazione

se un impiegato smette di partecipare a tutti i progetti, viene eliminato completamente dalla base di dati.

Anomalia di inserimento

non si può inserire un impiegato senza un progetto associato.

Perché è una situazione indesiderabile? Perché si rappresentano *nella stessa relazione* informazioni eterogenee: gli impiegati con i loro stipendi, i progetti con i loro budget, i ruoli degli impiegati nei progetti.

9.3 Dipendenze funzionali

Per studiare in modo sistematico questi aspetti si usa la **dipendenza funzionale**: un particolare vincolo di integrità del modello relazionale che descrive legami funzionali fra gli attributi di una relazione.

Definizione data una relazione r su schema $R(X)$ e due sottoinsiemi non vuoti Y e Z di X , esiste in r una dipendenza funzionale (FD) fra Y e Z , e si scrive $Y \rightarrow Z$, se per ogni coppia di tuple t_1, t_2 di r con gli stessi valori su Y , risulta che t_1 e t_2 hanno gli stessi valori anche su Z .

Esempio nella relazione precedente valgono:

$\text{Employee} \rightarrow \text{Wage} \quad \text{Project} \rightarrow \text{Budget} \quad \text{Employee Project} \rightarrow \text{Role}$

FD banali $\text{Employee Project} \rightarrow \text{Project}$ è “banale” (sempre vera). $Y \rightarrow A$ è **non banale** se A non compare in Y (se A è un insieme: nessun attributo di A appartiene a Y).

FD e anomalie le anomalie dipendono da alcune FD:

- gli impiegati hanno un unico stipendio: $\text{Employee} \rightarrow \text{Wage}$;
- i progetti hanno un unico budget: $\text{Project} \rightarrow \text{Budget}$.

Non tutte le FD causano anomalie: “in ogni progetto un impiegato ha un solo ruolo”, $\text{Employee Project} \rightarrow \text{Role}$, è soddisfatta “banalmente” perché Employee Project è una **chiave**.

- la terza FD ha a sinistra una chiave e *non* causa anomalie;
- le prime due non hanno a sinistra una chiave e *causano* anomalie;
- la relazione contiene informazioni legate alla chiave e altre legate ad attributi che non formano una chiave.

Le anomalie sono quindi causate dall’aver messo insieme concetti **eterogenei**: proprietà degli impiegati (stipendio), dei progetti (budget) e della chiave Employee Project (ruolo).

9.4 Forma normale di Boyce e Codd (BCNF)

Le forme normali sono proprietà soddisfatte solo in assenza di anomalie, definite tramite vincoli sulle dipendenze funzionali. La più importante è la forma normale di Boyce e Codd.

Definizione una relazione r è in **BCNF** se per ogni dipendenza funzionale (non banale) $X \rightarrow Y$ definita su r , X contiene una chiave K di r , cioè X è **superchiave** per r .

Normalizzazione se una relazione non è in BCNF, nella maggior parte dei casi si può sostituire con due o più relazioni normalizzate (in BCNF). Criterio: *se una relazione rappresenta più concetti indipendenti, va decomposta in relazioni più piccole, una per ogni concetto.*

Esempio di decomposizione la relazione delle anomalie diventa:

Employee	Wage
Jones	20
Smith	35
Williams	55
Brown	48
White	48

Project	Budget
Mars	2
Jupiter	15
Venus	15

Employee	Project	Role
Jones	Mars	Technician
Smith	Jupiter	Designer
Smith	Venus	Designer
Williams	Venus	Chief
Williams	Jupiter	Consultant
Williams	Mars	Consultant
Brown	Mars	Chief
Brown	Venus	Designer
White	Venus	Designer
White	Jupiter	Director

9.5 Decomposizione senza perdita

Una decomposizione con perdita data (Employee, Project, Office) con Employee $\rightarrow$ Office e Project $\rightarrow$ Office:

Employee	Project	Office
Jones	Mars	Rome
Smith	Jupiter	Milan
Smith	Venus	Milan
White	Saturn	Milan
White	Venus	Milan

la si decompone sulla base delle due FD in (Employee, Office) = {(Jones, Rome), (Smith, Milan), (White, Milan)} e (Project, Office) = {(Mars, Rome), (Jupiter, Milan), (Venus, Milan), (Saturn, Milan)}. Ricostruendo con il join naturale su Office si ottengono 7 tuple, fra cui le **spurie** (Smith, Saturn, Milan) e (White, Jupiter, Milan): il risultato è *diverso* dalla relazione originale.

Definizione una relazione r si decompone **senza perdita** su X e Y se il join delle proiezioni di r su X e Y è uguale a r :

$$\pi_X(r) \bowtie \pi_Y(r) = r$$

La proprietà è garantita se gli **attributi comuni** contengono una **chiave** di almeno una delle relazioni decomposte (nell'esempio, Office non è chiave né di una né dell'altra).

9.6 Conservazione delle dipendenze

Esempio: inserimento una decomposizione senza perdita si ottiene su Employee (chiave di (Employee, Office)): (Employee, Office) e (Employee, Project). Supponiamo di inserire la tupla (White, Mars, Milan): nelle relazioni decomposte non viola nulla, ma ricostruendo si ha Mars sia a Rome sia a Milan, violando Project $\rightarrow$ Office. Questa FD coinvolge attributi che non stanno più insieme in nessuna relazione, quindi non è più verificabile localmente.

Definizione una decomposizione **conserva le dipendenze** se ciascuna delle dipendenze funzionali dello schema originale coinvolge attributi che compaiono tutti insieme in uno degli schemi decomposti.

Proprietà richieste la normalizzazione per decomposizione deve garantire che gli schemi ottenuti, presi insieme, abbiano:

- la **decomposizione senza perdita**, che assicura la ricostruzione delle informazioni originarie;
- la **conservazione delle dipendenze**, che assicura il mantenimento dei vincoli di integrità originari.

9.7 Terza forma normale (3NF)

Una relazione non normalizzabile in BCNF

Chief	Project	Office
Smith	Mars	Rome
Johnson	Jupiter	Milan
Johnson	Mars	Milan
White	Saturn	Milan
White	Venus	Milan

Project Office $\rightarrow$ Chief
Chief $\rightarrow$ Office

Chief $\rightarrow$ Office viola la BCNF (Chief non è superchiave), ma Project Office $\rightarrow$ Chief coinvolge *tutti* gli attributi: nessuna decomposizione può conservarla. In alcuni casi la BCNF “non è raggiungibile”.

Definizione una relazione r è in **terza forma normale** se, per ogni FD non banale $X \rightarrow Y$ definita su r , è verificata almeno una delle condizioni:

- X contiene una chiave K di r (X è superchiave);
- ogni attributo in Y è contenuto in almeno una chiave di r .

Nell'esempio, Office fa parte della chiave (Project, Office): la relazione è in 3NF.

BCNF e 3NF

- la BCNF è più restrittiva della 3NF (la 3NF ammette relazioni con alcune anomalie);
- la 3NF è *sempre* raggiungibile (c'è un teorema), con decomposizione senza perdita e che conserva le dipendenze;
- se una relazione ha una sola chiave, è in BCNF se e solo se è in 3NF.

Decomposizione in 3NF

- si crea una relazione per ogni gruppo di attributi coinvolti in una dipendenza funzionale;
- si verifica che alla fine una relazione contenga una chiave della relazione originaria;
- il risultato dipende dalle dipendenze individuate.

Strategia: se la relazione non è normalizzata si decompone in 3NF, poi si verifica se lo schema ottenuto è anche in BCNF.

Una possibile riorganizzazione se nell'esempio si scopre un ulteriore concetto, il *dipartimento* (ogni dirigente dirige un dipartimento in una sede):

Dept. Office $\rightarrow$ Chief Chief $\rightarrow$ Office Dept. Project Office $\rightarrow$ Dept.

si ottiene una decomposizione in BCNF:

<u>Chief</u>	Office	Dept.
Smith	Rome	1
Johnson	Milan	1
White	Milan	2

<u>Project</u>	<u>Office</u>	Dept.
Mars	Rome	1
Jupiter	Milan	1
Mars	Milan	1
Saturn	Milan	2
Venus	Milan	2

9.8 Teoria delle dipendenze

I concetti visti si possono automatizzare: *data una relazione e un insieme di dipendenze funzionali su di essa, generare una decomposizione che contenga solo relazioni in forma normale e soddisfi le proprietà di decomposizione senza perdita e conservazione delle dipendenze.*

Implicazione dalle FD valide se ne possono derivare altre: un insieme di FD F **implica** f se ogni relazione che soddisfa tutte le dipendenze in F soddisfa anche f . Es.: Employee $\rightarrow$ Type e Type $\rightarrow$ Wage implicano Employee $\rightarrow$ Wage.

9.9 Progettazione e normalizzazione

La teoria della normalizzazione si può usare nella progettazione logica per verificare lo schema delle relazioni finali, ma anche nella progettazione concettuale per verificare la qualità dello schema concettuale.

Normalizzazione su entità *Product* (Code, Product Name, Price, Retailer Name, Address, VAT N) con la dipendenza

VAT N $\rightarrow$ RetailerName Address

viola la forma normale. Si decompone l'entità: *Product* (Code, Name, Price) (1,1) *Supply* (0,N) *Supplier* (VAT N, Name, Address).

Normalizzazione su relazioni la relazione ternaria *Thesis* fra *Professor* (0,N), *Student* (0,1), *Degree* (0,N) e *Dept.* (0,N), con le dipendenze

Student $\rightarrow$ Degree Student $\rightarrow$ Professor Professor $\rightarrow$ Dept.

non è in 3NF per Professor $\rightarrow$ Dept. (Professor non è superchiave, Dept. non appartiene a una chiave; la chiave è Student). Si decompone: *Work* fra *Professor* (1,1) e *Dept.* (0,N), e *Thesis* fra Professor, Student, Degree.

Ora *Thesis* è in BCNF rispetto a Student $\rightarrow$ Degree e Student $\rightarrow$ Professor; le due proprietà però sono *indipendenti* e si può decomporre ulteriormente: *Thesis* binaria fra *Professor* (0,N) e *Student* (0,1), ed *Enroll* fra *Student* (1,1) e *Degree* (0,N).

10 Basi di dati attive

10.1 Da passive ad attive

Le politiche di reazione dei vincoli di integrità sono il primo esempio di comportamento **reattivo** nelle basi di dati:

```
ON < DELETE | UPDATE >  
    < CASCADE | SET NULL | SET DEFAULT | NO ACTION >
```

L'idea è introdurre **costrutti di linguaggio** appositi, detti **regole** (attive), per gestire una parte del comportamento procedurale delle applicazioni. Stando a livello di base di dati, questo comportamento è “condiviso” fra le applicazioni (indipendenza dei dati).

Base di dati attiva ha una componente che gestisce regole attive del tipo **Evento-Condizione-Azione** (ECA):

- gli *eventi* sono modifiche della base di dati;
- si verifica una *condizione* (vero/falso);
- si eseguono una o più *azioni*.

Queste basi di dati hanno un comportamento *reattivo* (contrapposto a *passivo*): non eseguono solo le transazioni degli utenti, ma anche le regole. I DBMS commerciali (SQL-3) usano i **trigger**, definiti come le altre componenti dello schema.

10.2 Evoluzione

Anni '70: niente DBMS

le applicazioni $A_1, \dots, A_n$ condividono dati comuni direttamente sui file del sistema operativo.

Anni '80: primi DBMS

applicazioni (in Pascal, C) sopra un DBMS che gestisce le tabelle (sopra i file).

Anni '90: comportamento procedurale condiviso

nascono le **stored procedure**, per condividere comportamenti procedurali comuni fra applicazioni diverse. Non sono standardizzate e soffrono di *impedance mismatch* con il linguaggio usato per esprimerle; di conseguenza si introducono regole specifiche, i **trigger**, gestite direttamente dal DBMS (**DBMS attivi**).

Anni 2000

architettura client/server via Internet: front-end (JavaScript), back-end su web application server (Java), base di dati attiva con tabelle e trigger.

Anni 2010: app mobili

front-end in Java, Objective-C, HTML5; stessa struttura lato server.

10.3 Trigger

Si definiscono con istruzioni DDL (`CREATE TRIGGER`) e seguono il paradigma ECA:

- **evento**: modifica dei dati, specificata con `insert`, `delete`, `update`;
- **condizione**: predicato SQL (opzionale);
- **azione**: sequenza di istruzioni SQL (o estensioni, es. PL/SQL in Oracle).

Intuitivamente: quando accade l'evento (*attivazione*), se la condizione è soddisfatta (*valutazione*), allora si esegue l'azione (*esecuzione*). Ogni trigger fa riferimento a una tabella (*target*): risponde a eventi relativi a quella tabella.

Granularità

- **di tupla** (*row-level*): un'attivazione per ogni tupla coinvolta nell'operazione;
- **di operazione** (*statement-level*): una sola attivazione per ogni istruzione SQL, con riferimento a tutte le tuple coinvolte ("set-oriented").

Modalità

- **immediata**: subito dopo (o subito prima) dell'evento;
- **differita**: al momento del `commit`.

10.4 Modello computazionale

Sia $T^U = U_1; \dots; U_n$ la transazione dell'utente e P un insieme di regole della forma $E, C \rightarrow A$ (Evento, Condizione, Azione). Indicando con U_i^P la sequenza di azioni indotte da U_i su P :

- la semantica **immediata** genera $T^I = U_1; U_1^P; \dots; U_n; U_n^P$;
- la semantica **differita** genera $T^D = U_1; \dots; U_n; U_1^P; \dots; U_n^P$.

Problemi: **confluenza**, **terminazione**, **equivalenza**.

10.5 Trigger in Oracle

Sintassi

```
create trigger TriggerName
  Mode Event {, Event}
  on TargetTable
  [ [referencing Reference]
    for each row
    [when SQLPredicate] ]
  PL/SQLBlock
```

- *Mode*: before o after;
- *Event*: insert, update, delete;
- `for each row` specifica la granularità (di tupla);
- *Reference*: permette di definire nomi di variabili (solo per granularità di tupla): `old as OldVariable` | `new as NewVariable`.

Semantica

- modalità **immediata** (sia **after** sia **before**);
- piano di esecuzione: **trigger before statement**; per ogni tupla coinvolta: **trigger before row**, operazione, **trigger after row**; infine **trigger after statement**;
- in caso di errore si annulla tutto;
- priorità fra trigger;
- al massimo 32 trigger attivati in cascata.

Esempio riordino automatico in magazzino:

```
create trigger Reorder
after update of QtyAvbl on Warehouse
when (new.QtyAvbl < new.QtyLimit)
for each row
declare X number;
begin
  select count(*) into X
  from PendingOrders
  where Part = new.Part;
  if X = 0
  then
    insert into PendingOrders
    values (new.Part, new.QtyReord, sysdate);
  end if;
end;
```

Quando la quantità disponibile scende sotto la soglia, se non c'è già un ordine pendente per quel pezzo se ne inserisce uno.

Esempio 2 con Warehouse:

Part	QtyAvbl	QtyLimit	QtyReord
1	200	150	100
2	780	500	200
3	450	400	120

T1: update Warehouse set QtyAvbl = QtyAvbl - 70 where Part = 1

T2: update Warehouse set QtyAvbl = QtyAvbl - 60 where Part <= 3

- T1: pezzo 1 passa a $130 < 150 \Rightarrow$ si inserisce l'ordine (1, 100, oggi);
- T2: pezzo 1 passa a 70 (sotto soglia, ma c'è già un ordine pendente: nessun inserimento); pezzo 2 a 720 (sopra soglia); pezzo 3 a $390 < 400 \Rightarrow$ si inserisce (3, 120, oggi).

10.6 Trigger in DB2

Sintassi

```
create trigger TriggerName
  Mode Event on TargetTable
  [referencing Reference]
  for each Level
  [when (SQLPredicate)]
  SQLProceduralStatement
```

- *Mode*: before o after;
- *Event*: insert, update, delete;
- for each *Level* specifica la granularità (row o statement);
- *Reference*: nomi di variabili (in base alla granularità): old as *OldTupleVar* | new as *NewTupleVar* | old_table as *OldTableVar* | new_table as *NewTableVar*.

Semantica

- modalità **immediata** (sia after sia before);
- i trigger before non possono modificare la base di dati, se non per variazioni sulle modifiche causate dall'evento (quindi in generale non possono attivare altri trigger);
- in caso di errore si annulla tutto;
- nessuna priorità fra trigger (l'ordine è definito dal sistema); interazione con le azioni compensative dei vincoli di integrità referenziale;
- al massimo 16 trigger attivati in cascata.

Esempio

```
foreign key (Supplier)
  references Distributor
  on delete set null
```

```
create trigger SoleSupplier
  before update of Supplier on Part
  referencing new as N
  for each row
  when (N.Supplier is not null)
  signal sqlstate '70005' ('Cannot change supplier')
```

```
create trigger AuditPart
  after update on Part
  referencing old_table as OT
  for each statement
  insert into Audit
  values(user, current-date, (select count(*) from OT))
```

SoleSupplier impedisce di cambiare fornitore a un pezzo (ammesso solo porlo a NULL, come fa la on delete set null). AuditPart registra, per ogni istruzione di modifica, utente, data e numero di tuple modificate.

10.7 Estensioni

Di solito non disponibili:

- eventi temporali (anche periodici) o “definiti dall’utente”;
- combinazioni booleane di eventi;
- clausola **instead of**: non si esegue l’operazione che ha attivato l’evento, ma un’altra azione al suo posto;
- esecuzione “distaccata” (*detached*): si attiva una transazione autonoma;
- definizione di priorità;
- gruppi di regole attivabili e disattivabili;
- regole associate anche alle interrogazioni (non solo agli aggiornamenti).

10.8 Proprietà delle regole

- **terminazione** (essenziale);
- **confluenza**;
- **determinismo delle osservazioni**.

10.9 Applicazioni

- **funzionalità interne**: gestione dei vincoli di integrità, replicazione, gestione delle viste (materializzate: propagazione delle modifiche; virtuali: modifica delle interrogazioni);
- **funzionalità applicative**: descrizione del comportamento della base di dati (regole aziendali).